TECHNICAL UNIVERSITY OF SOFIA

FACULTY OF COMPUTER SYSTEMS AND TECHNOLOGIES

DESIGN AND IMPLEMENTATION OF A C++ COMPILE-TIME OBJECT-RELATIONAL MAPPING LIBRARY

DIPLOMA THESIS

Author:

Alex Tsvetanov, Faculty No. 121222225

Specialisation: Computer and Software Engineering

Supervisor:

Assoc. Prof. Eng. Galya Pavlova, Ph.D.

Sofia, 2026

CONTENTS

ABSTRACT

This diploma thesis presents the design, implementation, and evaluation of an Compile-time Object-Relational Mapping library for C++23 [12]. The work is organised around two complementary axes. The first is compile-time modelling of data and queries, in which the logical structure of data access is not described through strings interpreted at execution time, but through statically typed `constexpr` constructions. The second is a coroutine-based asynchronous architecture through which the same type-constrained model is extended toward non-blocking or pseudo-non-blocking execution according to the capabilities of the concrete backend driver.

The developed library uses a static data model expressed in C++ through constructs such as `property<T,Name>`, `relationship<...>`, and `table_name_trait<T>`, together with a typed intermediate representation of queries realised through `select_query`, `insert_query`, `update_query`, and `delete_query`. This logical layer is independent of the concrete database type. Materialisation toward relational and non-relational systems is performed by specialised `connector_trait<DB>` implementations, which declare supported capabilities and translate the same logical query into Structured Query Language (SQL), Binary JSON (BSON)-like filters, Redis commands, Cypher, or Cassandra Query Language (CQL) depending on the selected backend.

A significant extension of the architecture is its asynchronous layer, built on coroutines, `Task<T>`, `ThreadPool`, `async_db<DB>`, and coroutine-aware connection management. The thesis shows that a single asynchronous abstraction can be realised in two ways: by offloading blocking operations to a thread pool and by using native non-blocking interfaces for drivers that provide them. In this way, the work does not only describe a logical ORM abstraction, but also studies the execution model through which that abstraction remains practically useful under concurrent workloads.

The thesis analyses both relational scenarios for SQLite, PostgreSQL, and MySQL, and non-relational scenarios for MongoDB, Redis, Neo4j, and Cassandra. It demonstrates how the same C++ model dictates the logical schema while being materialised differently as a table, document, key-value structure, graph node, or wide-column row. It also examines the native client libraries and their execution semantics in order to determine where the library can honestly offer native asynchronous behaviour and where the correct choice remains a synchronous driver

with controlled offload execution.

Validation of the work is grounded in the repository's unit and integration tests, while the empirical evaluation is extended through benchmark scenarios comparing synchronous and asynchronous execution. These cover query construction, capability-based restriction of operations, prepared queries, migrations, thread safety, coroutine infrastructure, and the behaviour of concrete connectors with either live or mock drivers. This makes it possible to formulate not only an architectural model, but also concrete criteria for a "fully implemented and functioning" connector and for a practically justified asynchronous layer.

The main contribution of the thesis lies in four directions. First, it proposes a practically usable model for an Compile-time Object-Relational Mapping library that combines static type safety with a backend-independent architecture. Second, it shows how a C++ data model can serve as the primary source of a logical schema across different classes of databases. Third, it develops a coroutine-based asynchronous architecture that extends the connector model without compromising compile-time constraints. Fourth, it formalises a connector contract through which the library can be extended with new backend implementations and asynchronous capabilities without changing the user-facing query API.

Keywords: C++23, Compile-time Object-Relational Mapping, static type safety, coroutines, asynchronous execution, `connector_trait`, relational and non-relational databases, empirical evaluation

I. INTRODUCTION

1.1. Problem Statement

Data access remains one of the areas where the mismatch between the programming language and the storage model produces the greatest number of structural errors. In traditional database access layers, queries are described as strings and validated only at execution time. This means that incorrect column names, incompatible types, invalid `JOIN` constructs, or wrongly supplied parameters are discovered late — often against a real database and only after additional manual testing. At the same time, switching from one backend to another usually requires rewriting the entire data-access layer.

Object-relational mapping is the classical way of connecting an application's object model with persistent storage [4]. Most ORM implementations, however, remain strongly tied to runtime mechanisms: SQL strings, code generation, annotations, macros, or dynamic metadata descriptions. In the context of C++, this introduces an additional tension. The language provides a powerful type system, templates, `constexpr` evaluation, and `concepts`, yet database libraries rarely exploit these mechanisms far enough for the compiler to become a first-class validator of query structure and backend compatibility.

A distinctive feature of the present system is that, despite the name Compile-time Object-Relational Mapping library, its architecture is not restricted to the relational model alone. Instead, it uses the same C++ data model and the same query IR as a logical layer over relational and non-relational systems. This raises a second research question: which parts of the ORM abstraction are genuinely universal, and which parts must remain explicitly constrained through a capability-based backend contract.

The third layer of the problem concerns execution itself. Even when the logical model and query IR are type-correct, the practical value of a modern data-access library also depends on whether it can integrate asynchronous execution without breaking its type guarantees and without forcing the user back into a callback-based style. The thesis is therefore organised around two complementary axes: compile-time modelling and asynchronous execution architecture.

1.2. Objectives and Tasks

The primary objective of the thesis is to present the design and implementation of a Compile-time Object-Relational Mapping library for C++ that:

1. models data and queries through statically typed C++ constructs;
2. allows `SELECT`, `INSERT`, `UPDATE`, and `DELETE` queries to be built as `constexpr` objects;
3. separates logical query structure from backend-specific materialisation;
4. supports both relational and non-relational connectors through `connector_trait<DB>`;
5. uses compile-time capability tags to reject unsupported operations;
6. demonstrates how the C++ model determines both the logical schema and the physical data structure for different backend families;
7. realises a coroutine-based asynchronous layer that extends the synchronous connector model in a formal and type-safe way;
8. formalises a contract for adding new connectors, including cases where they support native asynchronous execution;
9. validates the architecture through systematic unit and integration tests and through empirical benchmark evaluation.

To achieve that objective, the work performs the following tasks: analysis of existing solutions; definition of a static entity model; construction of a typed query IR; design of the connector layer and capability mechanism; construction of a coroutine-based async layer; implementation of connectors for different classes of databases and different execution models; investigation of migrations, prepared queries, thread safety, and async connection management; formulation of extensibility criteria; and development of a benchmarking methodology for comparing synchronous and asynchronous execution.

1.3. Scope and Limitations

The thesis treats the library as a modelling, query-construction, and execution layer. Its scope therefore includes the entity model, query IR, placeholder mechanism, prepared queries, migration model, connector trait architecture, capability gating, the coroutine-based async layer, and the concrete backend implementations present in the repository. A substantial part of the evaluation is the scenario analysis of SQLite, PostgreSQL, MySQL, MongoDB, Redis, Neo4j, and Cassandra, together with benchmarking of synchronous and asynchronous execution modes.

Out of immediate scope remain production-scale optimisations such as distributed connection management, backend-complete migration strategies, live benchmarking across a full matrix of production environments, and full exploitation of future C++26 reflection. These are discussed as future work in Chapter X.

1.4. Thesis Structure

The thesis is organised as follows.

Chapter II reviews existing C++ solutions for database access and ORM, and positions the present work with respect to them.

Chapter III presents the theoretical foundations of Compile-time Object-Relational Mapping and the storage models required for the later relational and non-relational analysis.

Chapter IV introduces the theoretical apparatus of coroutines and asynchronous execution in C++, which is necessary for understanding the second major contribution of the library.

Chapter V describes the compile-time ORM architecture of the library: entity declarations, the query IR, `connector_trait`, capability mechanisms, and the place of migrations within the overall design.

Chapter VI presents the async execution architecture and shows how the coroutine-based layer connects to the synchronous connector model, the thread-pool strategy, and connection management.

Chapter VII analyses how the same logical abstraction and different execution strategies are materialised across relational, document, key-value, graph, and wide-column backends.

Chapter VIII brings together the verification evidence and the empirical benchmark evaluation.

Chapter IX formalises the contract for extending the library with new connectors, including capability and async requirements.

Chapter X outlines the main limitations and future development directions, while **Chapter XI** summarises the results and contributions.

II. REVIEW OF EXISTING SOLUTIONS

Reviewing existing solutions is necessary in order to evaluate whether the proposed architecture delivers a genuine contribution or merely reformulates already known practices. In the space of C++ database libraries, two principal axes and an additional execution perspective are especially useful: the level of abstraction, the stage at which queries are constructed and validated, and the execution model. The abstraction level ranges from almost direct use of a client C API to fully fledged ORM behaviour. The validation stage may be runtime, hybrid, or compile time. The execution model ranges from direct blocking API calls to backend-specific asynchronous state-machine interfaces and higher-level layers that attempt to unify them.

These axes are especially important in the context of the present thesis. If one observes only "API convenience", it is easy to miss the essential difference between a library that merely makes SQL strings more readable and a library that truly transfers structural correctness into the compiler. If, on the other hand, one looks only at when an error appears, one may underestimate the cost of abstraction, build-toolchain complexity, and suitability for more than one storage model. And if the execution model is ignored, it becomes easy to form the mistaken impression that "asynchrony" is a uniform capability regardless of whether one is dealing with a native non-blocking API, a callback layer over an event loop, or mere offload to a thread pool.

2.1. Classification of approaches

At the lowest level stand native client libraries such as the `SQLiteCAPI` [7], `libpq` [18], and `MySQLConnector/C` [15]. They provide maximal control over queries, parameters, and results, but offer no logical abstraction. The application must work directly with SQL strings, column indices, parameter positions, and backend-specific types.

The next group consists of runtime query-builder libraries. These typically improve readability and reduce part of the boilerplate, but they continue to rely on runtime construction of textual query representations. Type safety is only partial and usually stops at the API boundary. Structural validity of the query remains a runtime concern.

At the highest abstraction level lies the ORM approach. There, C++ classes are associated with tables, columns, and relationships, and the library generates the required queries. The benefit is a significant reduction in boilerplate code. The cost is that many such frameworks introduce external code generators, annotations, macros, or opaque runtime metadata, making

the correspondence between C++ code and the actual database operation harder to follow.

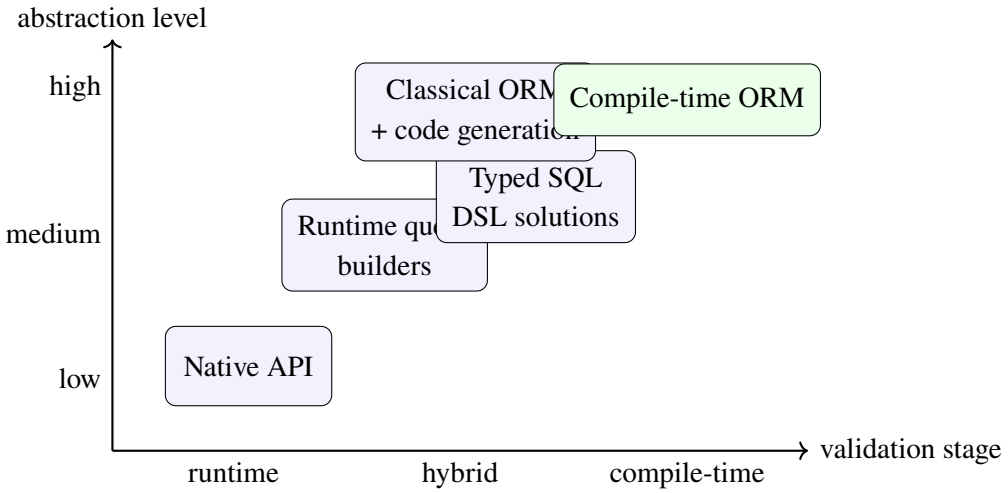

Figure 1. Positioning of the principal solution families by abstraction and validation stage

Figure 1 shows that the compile-time ORM approach is not merely “another ORM library”. It occupies the upper-right quadrant, where high abstraction is combined with early structural validation. That combination is rare because it demands greater architectural discipline inside the library itself.

Table 1. Comparison of major data-access approaches in C++

Approach	Abstraction level	Validation stage	Strength	Primary limitation
Native API	low	runtime	full control and closeness to the driver	strong coupling to a concrete backend
Runtime query builder	medium	mostly runtime	a more readable API and less boilerplate	the query still materialises as text
Typed SQL DSL	medium to high	partial compile-time	stronger static structure for the SQL world	usually remains limited to SQL
Classical ORM	high	runtime or code generation	convenient object-mapping behaviour	complex toolchain and reduced transparency
Compile-time ORM	high	compile time for structure	early discovery of structural errors and a formal connector contract	greater complexity in the template layer

2.2. Evaluation criteria

For the comparison to be academically complete, it is not enough to say that one library is “more convenient” while another is “lower level”. In the present work, six criteria are used:

1. **Level of logical abstraction** — to what degree the library allows the data model to be described independently of the immediate backend syntax.
2. **Degree of static validation** — which aspects of the model and query can be checked before execution.
3. **Portability across backends** — whether the logical model can be adapted to more than one storage family.
4. **Toolchain complexity** — whether the library requires external code generation, extra preprocessing stages, or generated metadata.

5. **Transparency of errors and execution** — how clearly a developer can follow the link between the C++ source, the generated query, and the runtime behaviour.

6. **Execution model and asynchronous suitability** — whether the library provides a unified and verifiable model for concurrent or non-blocking execution, or leaves that task entirely to the concrete driver and application code.

These criteria are directly related to the objectives of the thesis. If the proposed library is to be both expressive and formally checkable, then it must improve on alternatives not only in convenience, but in the balance between these six dimensions.

2.3. Native client libraries

SQLite [6], `libpq`, and `MySQLConnector/C` are representative of the approach in which the application constructs and submits queries directly to the driver. This is the most universal and often the most efficient option, but it places the full burden of correctness on application code. Developers must manually keep field names, parameter ordering, and schema details in sync. When the backend changes, not only the SQL dialect changes, but also the entire connection, preparation, binding, and result-reading API.

This becomes especially problematic when the same domain model must be connected to more than one class of database. In such cases the application accumulates conditional code paths, backend-specific SQL strings, and duplicated hydration logic. Beyond a certain system size, this weakens not only convenience but also confidence in correctness.

The strength of the native-API approach lies in its honesty. It does not promise more abstraction than it actually provides. Precisely for that reason, it often serves as a control point against which higher-level abstractions may be judged.

2.4. Asynchrony in existing client stacks

An essential part of the present context is that native client libraries are not always purely synchronous. `libpq` offers a state machine for asynchronous query dispatch and result consumption [19], MySQL Connector/C provides `_start`/`_cont` pairs for non-blocking operations [16], `hiredis` exposes a callback-based asynchronous layer [17], and the Cassandra driver relies on a future/callback mechanism [3]. Yet these capabilities remain strongly backend-specific both in interface and in execution semantics.

This leads to an important conclusion: the presence of a native asynchronous API does not by itself solve the problem of a shared C++ abstraction. In each separate case, the application must know whether it is working with socket-readiness polling, callback registration, future polling, or driver-managed internal threads. In MongoDB the emphasis remains on pooling and safe multithreaded use of `libmongoc` [14], while in `libneo4j-client` the dominant model remains synchronous request/response execution [13]. It is precisely this heterogeneity that motivates an intermediate asynchronous layer that is honest about backend differences and still unified enough for a user-facing API.

2.5. Runtime query builders

Libraries such as `SOCI` [8] and part of the lighter fluent SQL tooling seek a balance between readability and control. They offer a more convenient C++ interface for query construction, yet in many cases they still rely on runtime description and subsequent rendering to SQL. In practice this means that part of the query structure is better encapsulated, but backend-independent switching between fundamentally different storage models remains limited.

For the present work, these libraries are important as a point of comparison. They show that a fluent API and typed building blocks are useful, but not sufficient on their own. The decisive question is whether the compiler can validate the full logical structure of the query and whether the same abstraction can extend beyond the SQL world.

2.6. Typed SQL DSL solutions

`sqlpp11` [20] is particularly interesting because it stands between the traditional runtime query builder and the more strongly static design. Libraries of this family use templates and types to make part of the SQL structure visible to the compiler. This is a significant step beyond purely runtime builders, because part of the structural error space becomes visible earlier.

At the same time, however, a typed SQL DSL usually remains deeply tied to the relational model. Even when very strong as a C++ API, it typically assumes a world in which tables, columns, joins, and aggregates remain the central metaphor. That makes such solutions extremely valuable for SQL, but harder to generalise toward document, graph, or key-value systems.

This is exactly one of the main distinctions relative to the present work. The goal here

is not merely compile-time structuring of SQL, but a compile-time logical layer that can be materialised differently depending on the backend family.

2.7. Classical ORM frameworks and code generation

`ODB` [2] is a characteristic example of an ORM framework that provides rich mapping between classes and relational tables, but depends on code generation and an external toolchain. This makes the library convenient to consume, yet moves complexity into the build process and generated artefacts. A strong dependency appears between the data model, the additional tooling step, and the concrete relational backends.

A further tension is often visible in classical ORM frameworks: the higher the convenience at the application level, the less transparent the path toward the actual query and the performance characteristics of the system may become. This is not necessarily fatal, but it is an important limitation in the academic context of the present thesis, because it makes the relationship between abstraction and materialisation harder to inspect.

2.8. Limits of relationally centred abstractions

For applications that must work with document, graph, or key-value systems, the classical ORM model reveals natural limits. Some concepts — such as `JOIN`, foreign keys, and rigid tabular schemas — do not have direct equivalents in every storage model. Generalising ORM beyond the relational world therefore cannot be achieved merely by generating SQL.

This observation matters not only for classical ORM frameworks, but also for native APIs and typed SQL DSLs. Even when technically strong, they often assume that SQL remains the central language of the data world. The present work begins from a different premise: the logical model should be primary, while relational materialisation is only one possible interpretation.

Table 2. Limitations of the different solution families relative to the multi-backend objective

Solution family	What it does well	Where the limitation emerges
Native API	maximal control and driver-level closeness	no shared logical model and large duplication once more than one backend is used
Runtime builders	better readability and query encapsulation	validation remains mostly runtime and often SQL-centred
Typed SQL DSL	stronger static structure for SQL queries	the logic usually remains confined to the relational metaphor
Classical ORM	convenient object mapping for relational systems	dependence on code generation and difficult transfer to non-relational models

2.9. Positioning of the present work

The proposed library is positioned as a Compile-time Object-Relational Mapping solution that combines strong static typing, absence of external code generation, a connector-driven model for transferring one logical abstraction across different backend families, and a coroutine-based asynchronous execution layer. Compared to the native API approach, it reduces duplication of logic and the risk of late-discovered structural errors. Compared to runtime query builders, it shifts structural validation into the compiler. Compared to classical ORM frameworks, it avoids external generators and puts explicit emphasis on the connector contract. Compared to backend-specific asynchronous APIs, it offers a unified coroutine model without pretending that all drivers expose identical non-blocking capabilities.

The key distinction, however, is deeper. The present work does not treat compile-time techniques merely as a way to build a “smarter SQL builder”. It treats them as a way to formalise the logical model of data access itself. Analogously, it does not treat asynchrony as a decorative wrapper, but as a separate architectural problem with its own constraints, lifecycle rules, and backend-specific execution paths. That is precisely what allows the following chapters to move

from coroutine theory toward compile-time architecture, the async layer, connector contracts, and real multi-backend scenarios.

2.10. Conclusions from the comparative analysis

The comparison with existing solutions leads to four principal conclusions. First, the native-API approach remains indispensable as a baseline for control and performance, but does not provide a satisfactory level of logical abstraction. Second, runtime query builders and typed SQL DSL solutions improve work with Structured Query Language (SQL), but do not fully solve the problem of a backend-independent logical layer. Third, classical ORM frameworks offer high abstraction, but often at the price of tooling complexity and limited suitability outside the relational world. Fourth, native asynchronous client APIs are valuable, but they exhibit such different execution models that they do not by themselves provide a unified user abstraction.

It is exactly this positioning that justifies the more detailed discussion in the next chapter of a separate but crucial question: which language and theoretical mechanisms make a coroutine-based asynchronous architecture possible, on top of which the later compile-time ORM model and multi-backend execution layer are built.

III. THEORETICAL FOUNDATIONS OF COMPILE-TIME OBJECT-RELATIONAL MAPPING AND STORAGE MODELS

The theoretical basis of the present work is not reducible to a single language feature or a single database type. The library combines ideas from ORM theory, static typing in C++, compile-time construction of intermediate representations, and the comparative analysis of different storage models. To understand the architecture developed later, it is necessary first to separate the logical layer of data description from the physical layer of backend-specific materialisation.

Within this thesis, the term Compile-time Object-Relational Mapping should not be interpreted narrowly as a SQL-only technique. More precisely, it denotes an architectural approach in which the data model, the query form, and part of the admissibility constraints are described through the type system of the language rather than through runtime configuration. This makes it possible to carry one and the same logical structure toward relational, document, key-value, graph, and wide-column systems without hiding or falsifying backend-specific differences.

3.1. Taxonomy of the problem domain

Before the individual components are discussed, it is useful to fix the general picture of the problem domain. On one side stands the C++ model, in which the application describes entity types, fields, relationships, and constraints. On the other side stand different families of storage systems, each with its own model of physical organisation, access, and optimisation. Between them lies the typed query IR, which serves as the mediator between application logic and the concrete connector layer.

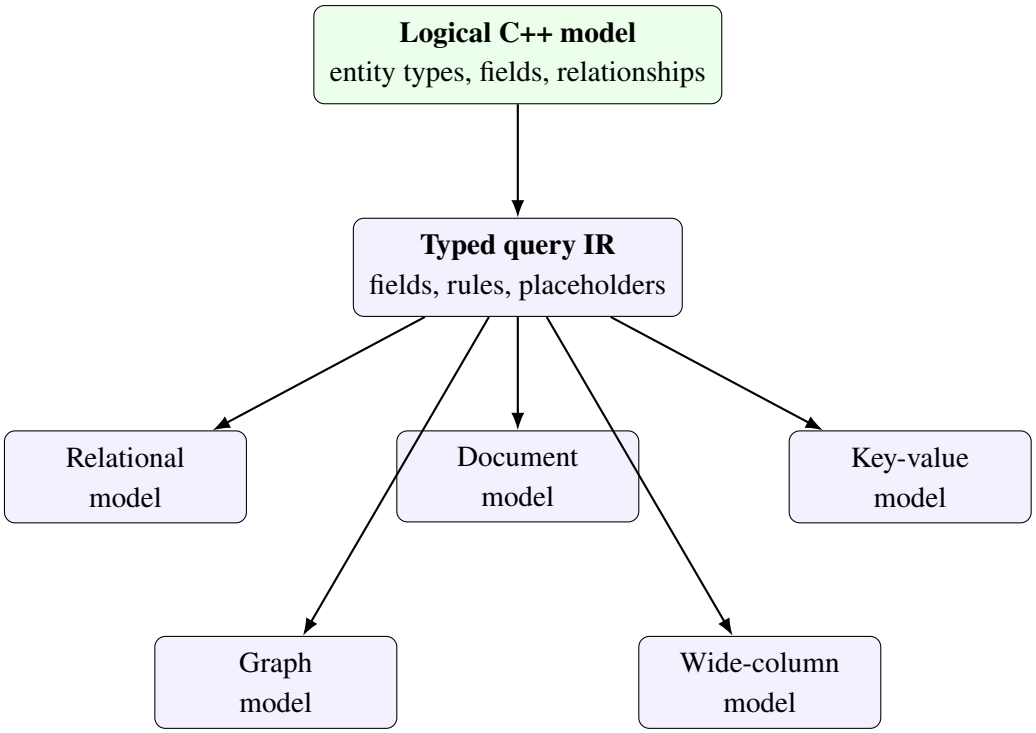

Figure 2. Taxonomy of the logical model, query IR, and backend families

Figure 2 highlights the central theoretical assumption of the library: what backends share is not their syntax, but the logical model of data and operations. A valid abstraction must therefore be positioned above SQL, BSON, Redis commands, Cypher, or CQL. It must operate at the level of fields, predicates, projections, relationships, and admissible operation classes.

3.2. Compile-time Object-Relational Mapping as an architectural approach

Classical ORM connects the object model of an application with its persistent schema [4]. In the compile-time variant, that connection is built not through runtime reflection or configuration files, but through types, templates, and `constexpr` values. In practice this means that field selection, admissible operations, and part of backend compatibility are checked by the compiler. Errors such as references to non-existent fields or attempts to use `JOIN` on a connector that lacks `supports_joins` cease to be runtime surprises.

An essential point is that compile-time ORM does not mean the absence of runtime data. Parameter values are still supplied at execution time, but the logical query form has already been fixed and validated. This yields a clear distinction between *logical structure* and *runtime values*. That distinction is fundamental for the thesis because it makes it possible to combine

static guarantees with real interaction against external databases.

From a theoretical perspective, this approach brings the ORM layer closer to a compiler pipeline. Application code describes a semantic structure, compile-time components turn that structure into an intermediate representation, and only the lowest layer materialises it into a backend-specific language or protocol. For that reason, the later architectural decisions in the library can be analysed through notions such as intermediate representation, static checking, and backend materialisation.

3.3. Static data modelling in C++

In the library, the domain model is expressed through C++ aggregates whose fields are declared using `property<T,Name>`. Each field thus carries both a value type and a symbolic field name. In addition, `table_name_trait<T>` links the type to a logical storage container — a table, collection, key prefix, label, or some other backend-specific notion.

This organisation has an important theoretical consequence: the C++ type becomes the primary carrier of the logical schema. Backends do not impose a separate user-level model; they merely interpret the same model differently. This is precisely why the same logical object may later become a relational table, a document with nested or referenced fields, a key-value structure, or a graph node.

Table 3. Mapping between C++ constructs and theoretical ORM concepts

Construct	Theoretical role	Practical effect in the library
`property<T,Name>`	logical-schema attribute	carries both value type and symbolic materialisation name
`relationship<...>`	logical object relationship	separates the relationship from the physical storage mechanism
`table_name_trait<T>`	logical container	serves as a table name, collection, key prefix, or label
`field<&T::m>`	attribute address in query space	connects the entity layer to the query IR
`connector_trait<DB>`	backend materialisation	defines how the logical model is executed physically

The ORM layer therefore does not begin from a driver or from SQL rendering. It begins from a static description of the meaning of the data. That is also why the entity layer is foundational for the entire architecture: if this layer is unclear or incomplete, no correct and extensible materialisation can emerge below it.

```cpp
template <typename T, detail::string_literal Name>
struct property
{
    using value_type = T;

    [[nodiscard]] static constexpr std::string_view column_name() noexcept
    {
        return Name.view();
    }

    T value{};
};

enum class store_as { reference, embed };

template <store_as Strategy, typename T, detail::string_literal Name>
struct relationship
{
    static constexpr store_as strategy = Strategy;
    using related_type = T;

    [[nodiscard]] static constexpr std::string_view field_name() noexcept
    {
        return Name.view();
    }
};
```

Listing 1: Excerpt from `property` and `relationship` in the entity layer

Listing 1 shows that the entity layer is minimalistic in form, yet semantically rich. `property` is not merely a value container; it also carries a name, and `relationship` is not merely a member field; it fixes the storage strategy as part of the type.

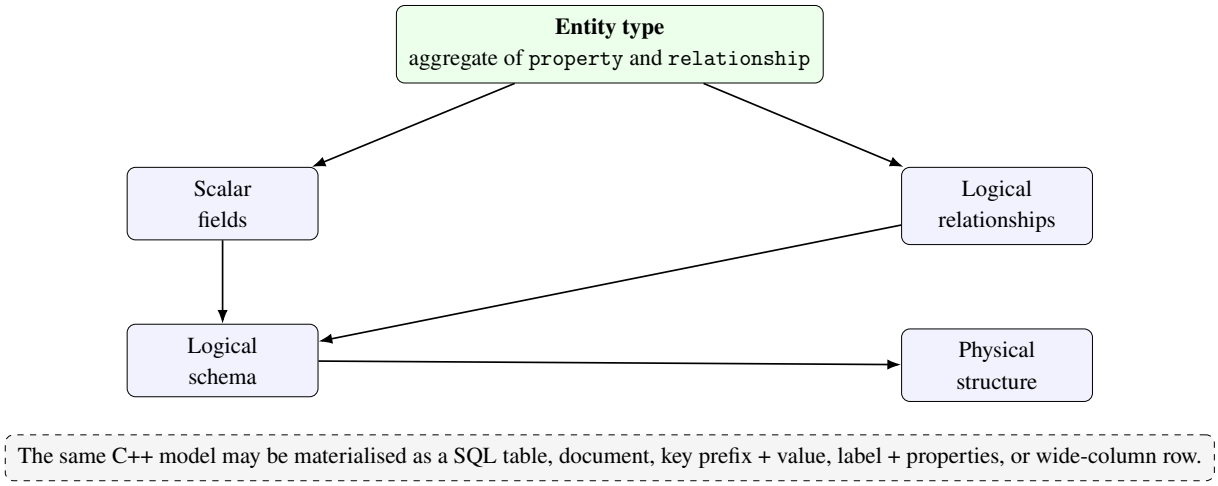

Figure 3. From C++ entity description to logical schema and physical materialisation

3.4. Typed query IR

Instead of describing a query directly as SQL or another textual representation, the library uses a typed query IR. Its building blocks are `field`, `Rule`, `Placeholder`, and query objects such as `select_query`, `insert_query`, `update_query`, and `delete_query`. This model offers two advantages. First, the query structure can be analysed at compile time and participate in `static_assert` checks. Second, the same IR can be rendered into different backend languages and protocols.

The IR therefore plays the role of an intermediate language between the domain model and the execution layer. The user works at a high level of abstraction, while the backend receives a specialised form aligned with its own storage model. This is why the query IR is logically closer to operation semantics than to the syntax of any concrete query language.

```
template <typename Response, typename Joins, typename Wheres,
          typename Limits, typename Groups, typename Orders>
struct select_query : select_query_tag
{
    using response_type = Response;
    using row_type = projected_type<Response>;

    template <typename RuleType>
        requires is_rule<RuleType>
    [[nodiscard]] constexpr auto where(RuleType r) const
    {
        using NewWheres = detail::append_type_t<Wheres, RuleType>;
        return select_query<Response, Joins, NewWheres, Limits, Groups, Orders>(
            response_, joins_,
            detail::tuple_cat(wheres_, detail::orm_tuple<RuleType>(r)),
            limits_, groups_, orders_);
```

```
    }
};
```

Listing 2: Excerpt from `select_query` and `where` clause composition

Listing 2 illustrates an important theoretical property: the query builder does not mutate text, but constructs new typed objects. This means that a sequence such as `select(...).where(...).group_by(...)` can be interpreted as the construction of a new type-level structure rather than as a series of string mutations.

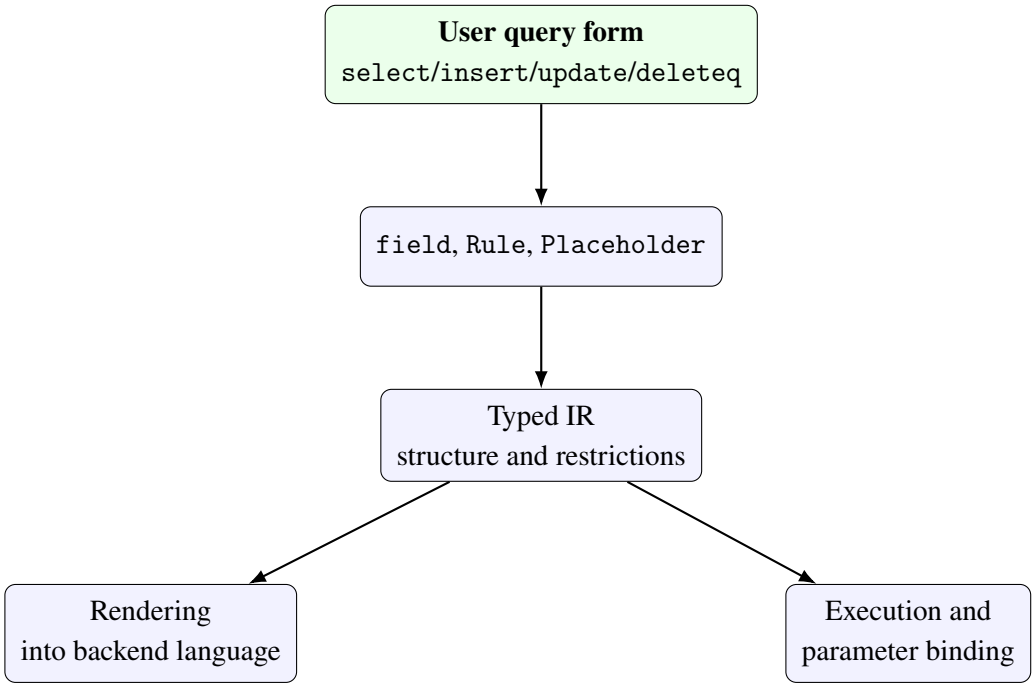

Figure 4. Typed query-IR pipeline from application API to backend execution

3.5. Relationship theory and storage strategies

Relationships between objects are among the hardest aspects of extending ORM beyond the relational world. In a relational context, a relationship is often materialised through a foreign key and a `JOIN`. In a document context, however, the same logical relationship may be realised as an embedded document, while in a key-value context it may require secondary lookup or denormalisation.

The library abstracts these options through `relationship<Strategy,T,Name>` and `store_as::reference/store_as::embed`. `reference` denotes a logical external reference, which the backend may realise through a foreign key, lookup, separate key, or graph edge. `embed` denotes inclusion of the related data in the same physical structure. The relationship is thus

described once at the logical level, while physical interpretation remains the connector's task.

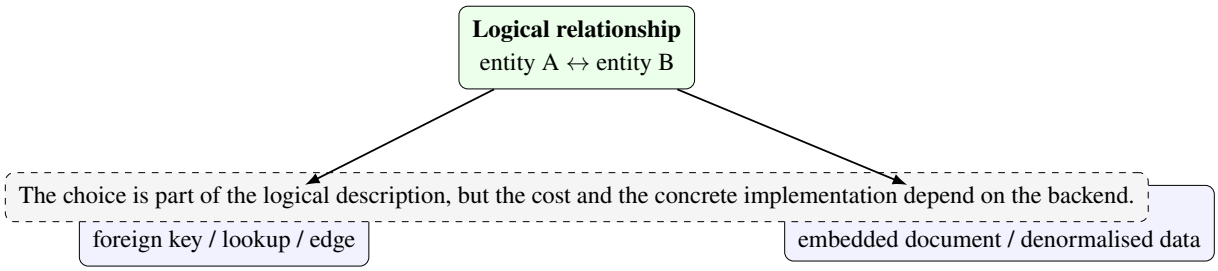

Figure 5. The two principal relationship materialisation strategies

Table 4. Comparison between `reference` and `embed` strategies

Strategy	Logical idea	Natural backend families	Primary trade-off
`reference`	separate identity of the related object	SQL, graph systems, key-based lookup	a separate navigation or joining mechanism is required
`embed`	physical inclusion in the parent structure	document systems, denormalised records	independent update and reuse become harder

3.6. Relational model

The relational model organises data into tables, rows, and columns. Its main strengths are a clear schema, a declarative query language, and formally defined integrity constraints. For ORM systems this is the natural environment: the properties of a C++ object map easily to columns, and relationships map naturally to foreign keys. Operations such as `JOIN`, `GROUPBY`, and transactions belong to the normal language model.

The relational backend is therefore the natural baseline scenario against which one can judge how far an ORM library preserves its usefulness outside the SQL world. In the present work, that baseline later materialises primarily through `SQLiteDB`, while portability of the IR is further analysed through `PostgreSQLDB` and `MySQLDB`.

3.7. Document model

The document model organises data into collections of documents whose fields may be nested and whose physical shape need not be identical across all records. This makes it naturally compatible with the `embed` strategy and more flexible for denormalised structures. On the other hand, directly projecting relational abstractions onto a document backend can be misleading. Not every relational operation has an equivalent with the same semantics or cost.

Within the present work, the document model is important as a test of whether the query IR expresses logic rather than the syntax of a particular language. If one and the same IR can be interpreted as a BSON-like filter and projection, without the user writing native document queries, then a genuinely storage-agnostic layer of abstraction has been achieved.

3.8. Key-value model

Key-value systems organise access around a key and its associated value. They are especially effective when the access pattern is explicit and centred around identifier-based lookup. Their disadvantage is that general predicate-based queries and relational operations are strongly limited. An ORM abstraction over a key-value backend therefore cannot promise the same expressive power as over a SQL system.

This model is useful for distinguishing between the *logical object* and the *admissible access pattern*. The object may remain the same, but not every query is meaningful against every physical organisation. In the repository, this becomes visible later through `RedisDB`, where lookup logic is natural, while join-oriented behaviour is architecturally inappropriate.

3.9. Graph model

The graph model represents data as nodes, edges, and properties. Its strength lies in expressing relationships as first-class objects rather than as values derived from foreign keys. This makes it suitable for scenarios involving rich navigation and traversal. At the same time, the graph model imposes its own query language and its own intuition for selection and filtering.

For the library, this is a revealing scenario because it demonstrates that `connector_trait` can transfer a shared query IR toward `MATCH/RETURN/WHERE` structures without forcing the user to write Cypher directly. If that is possible, then the common layer is indeed positioned above the concrete syntax layer.

3.10. Wide-column model

Wide-column systems such as Cassandra are based on distributed tables organised around partition and clustering keys. Although their syntax may resemble SQL, their access theory is different: a query is correct only if it follows the physical key model of data distribution. The mere existence of a field in the C++ model does not imply that it may be used freely in `WHERE` clauses.

This is exactly the type of limitation that justifies the library's capability- and constraint-based design. It is not enough for a query IR to be type-correct; it must also be admissible with respect to the storage model. For that reason, the wide-column model is especially valuable for the theoretical defence of compile-time constraints.

Table 5. Comparison of major storage models

Model	Storage unit	Typical relationships	Primary constraint
Relational	table/row	foreign key, join	consistent schema discipline and strong typing expectations
Document	document	embed or reference	not every relational operation is natural or inexpensive
Key-value	key and value	key-based lookup, hash fields	predicate queries outside the key path are strongly limited
Graph	node and edge	traversal relationships	specialised query language and graph semantics
Wide-column	partitioned row	key-oriented dependencies	queries must follow partition design rules

3.11. Test evidence behind the theoretical claims

The theoretical claims of this chapter are not detached from the code. The entity layer is validated directly in `tests/unit/test_property.cpp` and `tests/unit/test_relationship.cpp`, where column naming, entity recognition, the distinction between `store_as::reference` and `store_as::embed`, and one-to-many inference are checked. The typed query IR

is covered in `tests/unit/test_query.cpp`, which demonstrates field-based predicates, join forms, update composition, and delete semantics. In addition, `tests/unit/test_cpp26_reflection.cpp` shows that the compile-time description of fields can also be extended through a reflection-aware path when the toolchain supports it.

This relation between theory and test evidence is crucial. It shows that the presented notions are not free academic metaphors, but real architectural units that can be inspected in the source code and validated through automated tests.

3.12. Boundaries of the shared abstraction

An important conclusion follows from the above. A unified ORM abstraction does not mean that all backends become equivalent. Unification is possible at the level of logical model, field, relationship, query IR, and basic operation categories. Beyond that point, backend differences reappear: SQL `JOIN` has no universal meaning in Redis; embedding has no identical cost in a classical SQL table; graph traversal is not reducible to an ordinary `SELECT`; and the partition-driven restrictions of Cassandra cannot be masked as ordinary predicate rules.

For that reason, the architecturally correct strategy is not to hide all differences, but to make them explicit and formally controlled. That is precisely the role of `connector_trait`, capability tags, and compile-time restrictions, which are examined in the following chapters. There it becomes visible how the theoretical framework of the present chapter is turned into a working library architecture.

IV. THEORY OF COROUTINES AND ASYNCHRONOUS EXECUTION IN C++

Coroutines are a language mechanism for describing suspendable computations whose execution can be paused and later resumed without losing local state. In standard C++, coroutine support enters the language proper with C++20 rather than as an external library-level emulation [11, 10, 9]. This matters for the present work because it allows the asynchronous layer of the library to be built on compile-time defined rules for types, lifetime, and control flow instead of on an ad hoc callback structure.

In this chapter coroutines are examined not merely as convenient syntax, but as an execution model positioned between purely synchronous blocking execution and heavyweight operating-system threads. The goal is not only to show how `co_await` and `co_return` are used, but also to explain why compiler lowering to a state machine makes this approach especially suitable for a library that already relies on static types, trait specialisations, and compile-time validation.

This theoretical layer is necessary for one more reason. In the context of Compile-time Object-Relational Mapping, coroutines are not an isolated language experiment, but the foundation of a unified asynchronous interface over backends with very different operational characteristics. For such an architecture to be scientifically defensible, the language mechanism, the runtime infrastructure, and the concrete driver execution semantics must be kept distinct.

4.1. Processes, threads, tasks, and coroutines

A process is a protected address-space context provided by the operating system, a thread is a kernel-scheduled sequence of instructions, and a task is a more general software notion for a unit of work. A coroutine is neither a process nor a thread. It is a cooperative execution unit whose suspension and resumption are governed by the programming model and the generated runtime structure rather than directly by the operating-system scheduler.

This distinction is essential because the library combines coroutines and `ThreadPool` without conflating the two abstractions. A thread is a comparatively expensive kernel resource associated with its own stack, scheduling, and synchronisation primitives. A coroutine is a much lighter logical unit that preserves only the state required to suspend and continue a concrete computation. Moving toward a coroutine-based model therefore does not mean “more threads”,

but a more structured way of describing where a computation may wait for an external event.

Table 6. Comparison of the principal execution units

Unit	Nature	Significance for the library
Process	Isolated address-space context with operating-system resources	Not a direct ORM architectural unit, but the boundary of the runtime environment
Thread	Kernel-scheduled execution flow with its own stack	Used by `ThreadPool` for offloading blocking operations
Coroutine	Cooperatively suspendable computation with preserved local state	Provides the formal model for `Task<T>` and asynchronous composition

Table 6 underlines that a coroutine is a logical rather than a kernel unit. That is exactly what allows many suspend/resume sequences to coexist within the same thread without each requiring its own stack and thread identity.

4.2. Syntax, awaitable protocol, and language bindings

The key language operators `co_await`, `co_return`, and `co_yield` mark that a function follows coroutine semantics. From the standpoint of the architecture, the most important one is `co_await`, because it describes the point at which the current computation may yield execution until an external operation — for example a database call or scheduling to a thread pool — becomes ready. In this way the user-facing syntax remains close to linear code while the execution model becomes asynchronous.

For the purposes of the present thesis, it is important that this language form is not discovered dynamically at runtime, but is compiler-bound to the function's return type and to the `promise_type` that `std::coroutine_traits` derives for the concrete signature [11, 10]. This means that the way a coroutine produces a result, propagates an error, or stores its continuation

can be formalised already at the level of the type system.

In practice, `co_await` operates through the awaitable protocol. The object being awaited participates in a three-step scheme: `await_ready()` determines whether suspension is needed at all, `await_suspend()` receives the coroutine handle and decides how resumption should be organised, and `await_resume()` returns the result or raises an error. It is precisely this triple that makes it possible for different sources of asynchrony — thread-pool tasks, event-loop readiness, and driver callbacks — to be unified under a common `co_await` syntax.

4.3. The coroutine as a state machine

The compiler does not perform "magic", but lowers a coroutine function into a state machine with well-defined suspend points, resume transitions, and frame storage [10, 9]. This transformation is especially important for the present thesis because it shows that the compile-time approach of the library does not stop at query modelling. The same language toolchain also makes it possible to formalise asynchronous control flow through types and static rules instead of through unstructured callback chains.

Conceptually, a coroutine function passes through several clearly distinguishable phases: creation of the frame and `promise_object`, initial execution, arrival at a suspend point, externally organised resumption, and final completion. This matters because correctness no longer depends only on "what the code does", but also on when it is admissible to continue it and which component bears responsibility for that continuation.

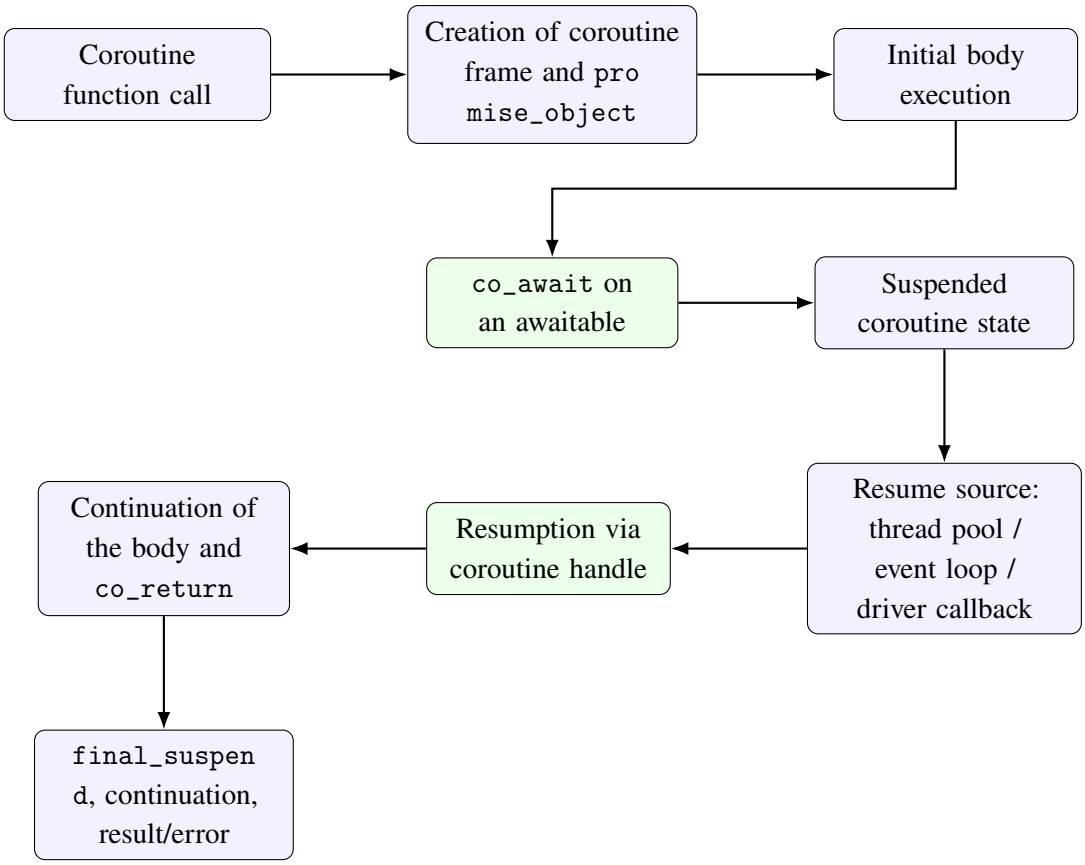

Figure 6. Conceptual coroutine lifecycle as frame, suspend points, and externally controlled resumption

Figure 6 shows why the coroutine model is natural for a library with clearly separated layers. The user's logical code remains linear, while the actual control of execution passes through formal transitions that can be connected to concrete runtime mechanisms.

4.4. Coroutine frame, promise object, continuation, and safety

For each coroutine the compiler creates a coroutine frame that stores local variables, the current execution state, and the promise object through which the coroutine communicates a result, an error, or completion. This has a direct impact on safety: the lifetime of the frame, the ownership of the result, and the placement of the continuation are not secondary implementation details, but part of the correctness contract of the asynchronous layer.

It must be emphasised that the coroutine frame is not an ordinary stack frame. It may outlive the current stack context and be resumed later from another thread or from event-loop infrastructure. Problems such as premature destruction of the frame, double resumption, incorrect propagation of exception state, or loss of continuation are therefore not mere low-level implementation details, but fundamental correctness risks.

The promise object is precisely what organises value return, error propagation, and the behaviour at `initial_suspend` and `final_suspend`. From an architectural perspective, this is an important example of compile-time binding: the result type and completion semantics are not determined by a dynamic registry scheme, but encoded in the coroutine return type itself. That allows `Task<T>` to be designed as a formal abstraction rather than as an informal "future-like" object.

4.5. Coroutines, thread pools, and event-loop infrastructure

`ThreadPool` and coroutines solve different problems. `ThreadPool` provides a bounded set of worker threads and a scheduling policy for work, whereas coroutines describe how a logical task may be suspended and later resumed. Their combination allows blocking backends to be integrated in a universal way through offload, without forcing the user back into callback-oriented programming.

On the other hand, with truly non-blocking drivers the coroutine may be resumed not by a worker thread, but by an event loop or a callback bridge. Reactor-like environments observe file-descriptor readiness through system mechanisms such as `kqueue` on macOS and BSD systems [1]. The coroutine therefore does not itself "perform I/O"; rather, it provides a formal interface through which I/O readiness or a driver callback can be turned into a resume action.

This is exactly where it becomes clear why coroutines should not be conflated with threads. The thread is the carrier of physical execution; the coroutine is the carrier of logical sequencing. One and the same asynchronous model may use a thread pool in one backend scenario and an event loop in another without changing the user-facing `co_await` syntax.

4.6. Significance for the Compile-time Object-Relational Mapping library

For Compile-time Object-Relational Mapping, this theoretical analysis has a direct architectural consequence. The compile-time modelled query IR and connector contract determine what is admissible to execute; the coroutine-based layer determines how that admissible computation may be executed asynchronously. In other words, a type-correct query and a correct execution path are distinct but mutually dependent layers.

This distinction is especially important in a multi-backend library. PostgreSQL and MySQL provide native non-blocking client interfaces, but in different operational forms [19, 16]. `hiredis` uses a callback-based asynchronous layer [17], while the Cassandra driver operates through a future/callback mechanism [3]. In MongoDB the emphasis remains on safe pooling and multithreaded use of `libmongoc` [14], whereas `libneo4j-client` follows a predominantly synchronous request/response model [13]. It is precisely this heterogeneity that justifies an asynchronous architecture with two strategies: a universal offload model and native async integration for drivers that permit it.

Coroutines therefore do not automatically make every operation non-blocking and they do not remove the limitations of a given driver. They provide a formal model for asynchronously describing the computation; the actual performance effect depends on whether the backend library has a native non-blocking interface, whether work is offloaded to a thread pool, or whether the task remains synchronous. This is exactly why the following chapters consider separately the compile-time ORM architecture, the asynchronous layer of the framework, and the execution semantics of concrete backends.

V. COMPILE-TIME ORM ARCHITECTURE

The compile-time ORM architecture of the library is organised around a clear separation between the logical data model, the intermediate representation of queries, and the backend-specific materialisation layer. This layer carries the first principal scientific axis of the thesis: how the compiler can participate in modelling, constraining, and validating data access before the runtime phase begins. To achieve this, the architecture is built from several layers, each with its own responsibility and with strictly delimited interfaces.

The theoretical chapter showed that unification is possible at the level of the logical model and the query IR, but not by ignoring the differences between backend families. After the coroutine-theory chapter, the focus here deliberately returns to the compile-time core of the system: entity descriptions, the query IR, `connector_trait<DB>`, and the capability mechanism. The library is therefore treated not merely as an idea, but as an organised system of types, contracts, and formal boundaries on admissible operations.

5.1. Architectural goals and constraints

The architecture was designed under several constraints acting simultaneously. First, the user-facing API must remain within idiomatic C++, without introducing a separate DSL and without relying on runtime reflection. Second, the query structure must be representable as a `constexpr` object so that a significant portion of validation can be performed by the compiler. Third, the backend layer must remain extensible without inheritance from a common virtual interface, because such an interface would hide important backend differences and move correctness checks toward runtime.

A fourth constraint follows from the multi-backend nature of the system itself. The library must support SQL and non-SQL connectors without false equivalence. This means that the architecture must simultaneously provide a shared API and express the limitations of a concrete backend in a formal and testable manner. The choice of a trait-based connector layer together with a capability mechanism follows naturally from this requirement.

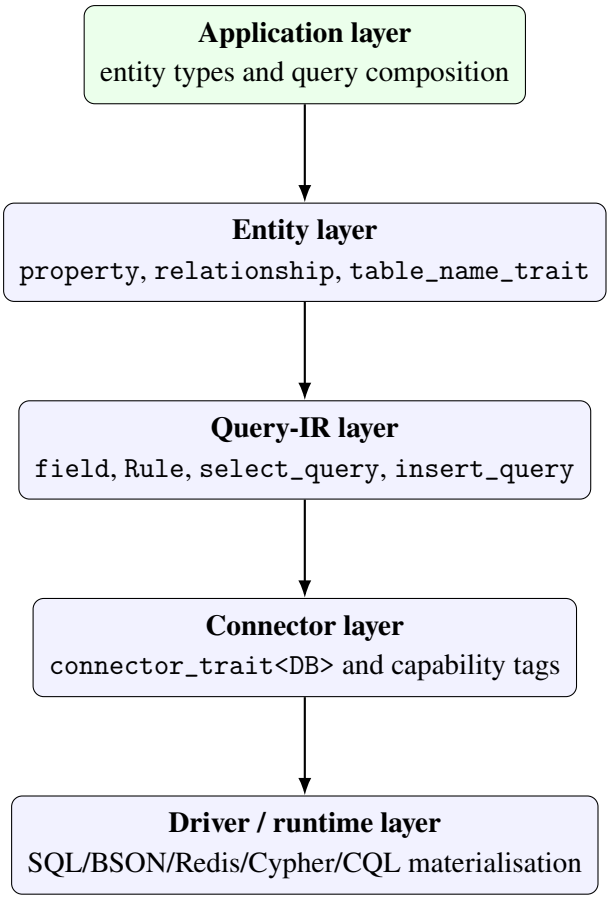

Figure 7. Layered architecture of the library

Figure 7 shows that the library follows a consistent vertical organisation. Application code does not communicate directly with the driver; it works with C++ types and query builders. The driver, in turn, does not know domain semantics, but only the materialised form generated by the connector layer. This is precisely why the architecture can remain both strict and extensible.

5.2. Layered organisation and responsibility separation

The topmost layer is the application code, which defines entity types, constructs queries through `select`, `insert`, `update`, and `deleteq`, and submits them to `db<DB>`. The next layer is the logical model of the library — the entity descriptions and the query IR. The third layer is the connector abstraction, where `connector_trait<DB>` materialises the same logic into backend-specific syntax and behaviour. At the bottom stands the concrete driver, wire protocol, or mock implementation.

This architecture has an important practical effect. If the user replaces `SQLiteDB` with `MongoDB`, the query IR remains logically the same, but the connector interprets it as filter/projection execution rather than SQL rendering. This does not mean that every IR is admissible on every

backend; it means that the *form of abstraction* is shared, while admissibility is determined formally through the capability mechanism.

Table 7. Principal architectural layers and their responsibilities

Layer	Main artefacts	Responsibility
Application	entity types, query-builder calls	describes domain logic and formulates operations in idiomatic C++
Logical	`property`, `relationship`, `field`, `Rule`	models data and query structure independently of a concrete backend
Connector	`connector_trait<DB>`, capability tags	checks admissibility and materialises the IR into backend behaviour
Runtime/driver	concrete connection handles and driver APIs	executes the materialised query and returns results

Table 7 shows that the library does not rely on a single monolithic object that does everything. Instead, decisions are localised in separate layers. This matters both for extensibility and for academic analysis, because each claim can be traced to concrete types and modules.

5.3. The entity layer as architectural foundation

The entity layer is built around `property<T,Name>`, `relationship<...>`, and `table_name_trait<T>`. It plays a dual role. On the one hand, it represents the C++ domain model. On the other hand, it serves as a logical description of the persistent schema or structure. This means that the architecture does not construct a separate runtime metadata registry. Instead, the type system contains enough information to derive field names, value types, relationships, and the target storage container.

Especially important is the fact that the entity layer is independent of the database family. Neither `property` nor `relationship` is SQL-specific. They describe logical attributes and logical relations. This creates the foundation for comparable behaviour across relational and non-relational backends. At the architectural level, this is exactly what makes it possible for the thesis to argue that the C++ model is the primary carrier of the logical schema.

5.4. Query IR as the central architectural component

The query IR is the central architectural component. It combines field references, predicates, placeholders, and additional clauses such as `order_by`, `group_by`, and `limit`. Instead of working with textual queries, the library constructs typed objects whose template parameters encode the logical structure of the operation.

From the architectural point of view, this has several consequences. First, structural validation occurs early. Second, the query IR can be analysed by the connector layer without losing semantic information. Third, prepared queries can preserve not an SQL string, but the intermediate representation itself, which is more natural for a library oriented around the C++ type system.

Fourth, the query IR is the single zone in which entity description and later backend materialisation meet. In that sense it is the architectural hinge of the whole library: high enough to remain storage-agnostic, yet concrete enough to be executed.

5.5. The connector layer and the formal contract

Formally, the connector layer is implemented through specialisation of `connector_trait<DB>`. This approach is analogous to standard trait templates such as `std::iterator_traits`. The type `DB` may be a simple wrapper over a connection handle, while the entire ORM logic remains in the trait specialisation. In this way, the library does not require the user or the connector author to inherit from a common base class.

`connector_trait<DB>` defines at least three central aspects: `wire_type`, `cursor_type`, and `execute(...)`. The specialisation may also declare capability tags such as `supports_joins`, `supports_transactions`, `supports_aggregation`, or `supports_concurrent_execute`. The presence or absence of these pseudo-markers is sufficient for the upper layers to impose compile-time restrictions.

```
template <typename DB>
concept is_connector = requires {
    typename connector_trait<DB>::template wire_type<int>::type;
    typename connector_trait<DB>::cursor_type;
};

template <typename DB, typename Cap>
inline constexpr bool has_capability =
    detail::capability_check<DB, Cap>::value;
```

Listing 3: Excerpt from the formal connector contract

Listing 3 shows that the contract is structural rather than inheritance-based. This is a decisive architectural choice. If a virtual interface existed with runtime dispatch, some of the type-checkable information would be lost. The library instead prefers a compile-time structure in which the requirements remain visible both to the compiler and to the connector author.

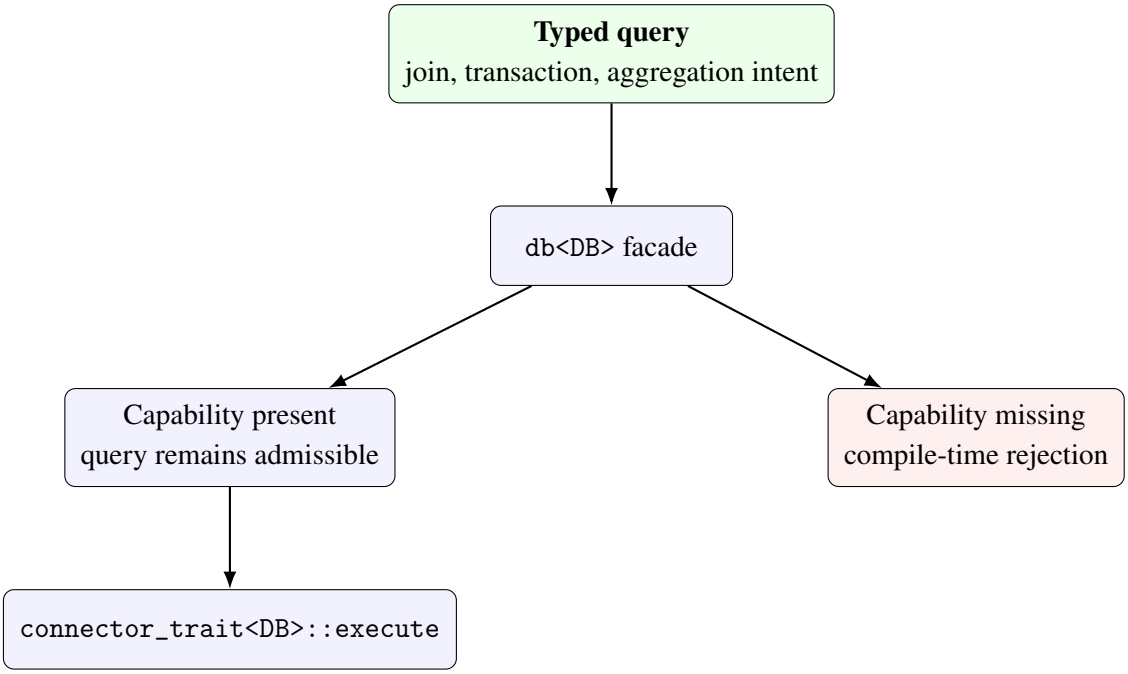

Figure 8. Capability gating between the query IR and the connector layer

Figure 8 is important because it shows where the architecture decides admissibility. This is not the driver and not a runtime log. The decision is taken in the public API layer, which already has access both to the query IR and to the trait-level information about the connector.

Table 8. Role of the principal capability tags in the architecture

Capability	Architectural meaning
`supports_joins`	admits join-based query forms for the selected backend
`supports_transactions`	allows use of `transaction_guard` together with `begin/commit/rollback` hooks
`supports_aggregation`	marks support for aggregation operations in the logical query layer
`supports_embedding`	makes native support for embedded structures explicit
`supports_upsert`	marks backends for which upsert belongs to the natural execution contract
`supports_bulk_insert`	denotes the existence of a more efficient bulk-write path

5.6. The user-facing db<DB> facade

The class `db<DB>` is the public facade layer. It encapsulates a reference to a concrete backend object and provides a unified execution API. In practice, this layer has two roles. The first is convenience for the user: the library is used through the same interface regardless of backend. The second is architectural filtering: this is precisely where compile-time checks for capabilities and backend compatibility are performed.

Consequently, `db<DB>` is not merely a thin wrapper over `execute`. It is the place where the library formalises the boundary between admissible and inadmissible behaviour for the selected backend. It is exactly this layer that enables error messages such as “this connector does not support `JOIN` operations” instead of allowing such errors to surface later as ambiguous runtime behaviour.

```
template <typename DB>
    requires is_connector<DB>
class db
{
public:
    template <typename Query>
    auto operator<<(Query q)
    {
        if constexpr (is_select_query<Query>)
        {
```

```
            if constexpr (decltype(q.join_clauses())::size > 0)
            {
                static_assert(has_capability<DB, cap::supports_joins>,
                    "orm::db: this connector does not support JOIN operations.");
            }
        }
        return connector_trait<DB>::execute(*conn_, std::move(q));
    }
};
```

Listing 4: Excerpt from the db<DB> facade layer

Listing 4 shows why db<DB> is more than a convenient facade. It is the formal location where query structure and connector capability information meet. This is where the compile-time architecture becomes a real limitation on API use.

5.7. Migrations as an architectural extension

The architecture also includes a prototype migration layer through migrate<DB>, live_schema, entity_meta, and DDL operations such as create_table_op, add_column_op, drop_column_op, and alter_column_type_op. This layer is particularly important from the standpoint of the thesis because it makes explicit the relation between the C++ model and the real storage schema.

The migration layer is not uniformly complete for every backend, but architecturally it demonstrates that the library is not limited to query execution. With a sufficiently rich connector_trait<DB> specialisation, entity description can also participate in schema evolution. This changes the very meaning of the ORM layer: it becomes not only an interface to data, but a potential source of schema-aware tooling.

5.8. Implementation of the principal subsystems

Once the architectural model has been outlined, the next question is how it is materialised in concrete C++ constructs and how the subsystems cooperate inside the repository. This section restores the missing engineering perspective: not merely which layers exist, but how where clauses are accumulated, how a query object is preserved for reuse, how migrate<DB> turns the difference between entity description and live_schema into DDL operations, and how the helper layer enforces rules for thread safety and transaction handling.

This section also fixes the working notation without reintroducing a standalone glossary

chapter. Here, DB denotes a generic backend type, db<DB> denotes the public execution facade, connector_trait<DB> is the formal connector contract, field<...> is the compile-time representation of a field, Placeholder<T> is an anonymous runtime parameter, and orm::ph<T,_N> is an indexed placeholder. The repository context for this mechanics is concentrated mainly in lib/include/ORM/entity/, lib/include/ORM/query/, lib/include/ORM/connector/, lib/include/ORM/db/migration/, and in the verification layer under tests/unit/ and tests/integration/.

5.8.1. Mini case study with User and Post

To keep the exposition directly tied to the real code, this section uses the User and Post types from tests/integration/test_mockdb.cpp as a supporting mini case study. They are especially useful because they combine scalar fields, explicit naming of logical containers, and a join scenario. The same example is later reused for placeholder binding, prepared-query behaviour, and SQL rendering assertions.

```
struct Post
{
    orm::property<int, "id">              id;
    orm::property<int, "author_id">       author_id;
    orm::property<std::u8string, "body"> body;
};

struct User
{
    orm::property<int, "id">             id;
    orm::property<std::u8string, "name"> name;
    orm::property<double, "score">       score;
};
```

Listing 5: Entity types from tests/integration/test_mockdb.cpp

Listing 5 shows not only how the entity model is described, but also how it participates directly in the tests. table_name_trait<User> and table_name_trait<Post> fix the logical containers "users" and "posts", while member pointers such as &User::id and &Post::author_id later participate in the query builder. This is a direct illustration of one of the central motifs of the thesis: the model, the query form, and the execution tests all use the same compile-time artefacts.

5.8.2. Scalar properties and naming of containers

The implementation of `property<T,Name>` binds a value type to a compile-time name. The function `column_name()` gives unambiguous access to the symbolic field name and is used by connectors when rendering toward SQL columns, document fields, key-space segments, or other backend-specific notions. The construction is minimalistic but sufficient because it avoids any external runtime metadata registry.

Naming of the logical storage container is implemented through `table_name_trait<T>`. Although the name is inherited from ORM tradition, architecturally it plays a more general role: in SQLite, PostgreSQL, and MySQL it means a table, in MongoDB a collection, in Redis a key prefix, and in Neo4j a label. In this way the library uses a unified logical interface for addressing physically different structures.

The practical consequence matters: a connector author does not derive schema information from an external registry, but from the type system. This reduces boilerplate and makes naming mistakes easier to localise both at compile time and in unit tests.

5.8.3. Field wrappers, rules, and placeholders

The field in a query expression is represented by `field<&T::m>`. This is a `constexpr` wrapper over a member pointer that carries enough information about the container type, the value type, and the column name. Operators over `field` construct typed `Rule` objects. Expressions such as `field<&User::id>==Placeholder<int>\{\}` are therefore not strings, but compile-time descriptions of predicates.

The placeholder mechanism has two forms. `Placeholder<T>` models anonymous parameters that are consumed positionally by the arguments of `execute()`. `orm::ph<T,std::placeholders::_N>` models indexed parameters that allow one runtime value to be reused in more than one position. This is especially important for backends such as SQLite, PostgreSQL, and MySQL, where the placeholder slot is a distinct part of the materialisation logic.

The tests `IndexedPlaceholderRendersQuestionMarkN`, `IndexedPlaceholderReusedSameArgInSQL`, and `TwoDistinctIndexedPlaceholdersRenderCorrectSlots` in `tests/integration/test_mockdb.cpp` show that this subsystem is not a theoretical addition, but a real mechanism for expressing more precise parameter-binding scenarios.

5.8.4. Construction of query objects

The factory functions `select`, `insert`, `update`, and `deleteq` create query objects upon which `where`, `join`, `order_by`, `group_by`, and `limit` are layered in sequence. Crucially, each of these operations does not modify a string, but returns a new typed object with extended internal state. As a result, the query builder remains fluent in appearance yet static in nature.

This allows the tests to validate not only the final rendering result, but also the form of the intermediate representation itself — for example the number of `where` clauses, the response-tuple type, or the correctness of `GROUPBY` and `ORDERBY` composition. In `tests/integration/test_mockdb.cpp` this is visible in `SelectWithInnerJoin`, `SelectWithOrderByDesc`, `GroupByMultipleFields`, `SelectWithLimit`, and numerous other scenarios that validate different phases of query composition.

One key implementation detail is that the builder chain remains `constexpr`-compatible. The internal helper `orm::detail::tuple_cat` uses pack expansion through index sequences rather than reliance on runtime concatenation. The practical consequence is that a significant part of the query shape can be built and validated already during compilation.

```
using namespace orm::literals;
static constexpr auto get_active =
    orm::select(orm::field<&User::id>, orm::field<&User::name>)
        .where(orm::field<&User::score> > 0.0)
        .where(orm::field<&User::id> == orm::Placeholder<int>{})
        .limit(10_per_page & 2_page);
```

Listing 6: Query object declared as a compile-time constant

The operator `&` combines the `_per_page` and `_page` helpers into a `Pagification` object whose structure is known at compile time. The header `lib/include/ORM/query/limits.hpp` also supports a symmetric `*` form, but the use of `&` in Listing 6 is more readable for the query style of the library. In this way the query can be declared as `staticconstexpr` and reused without reconstructing the builder form at every call.

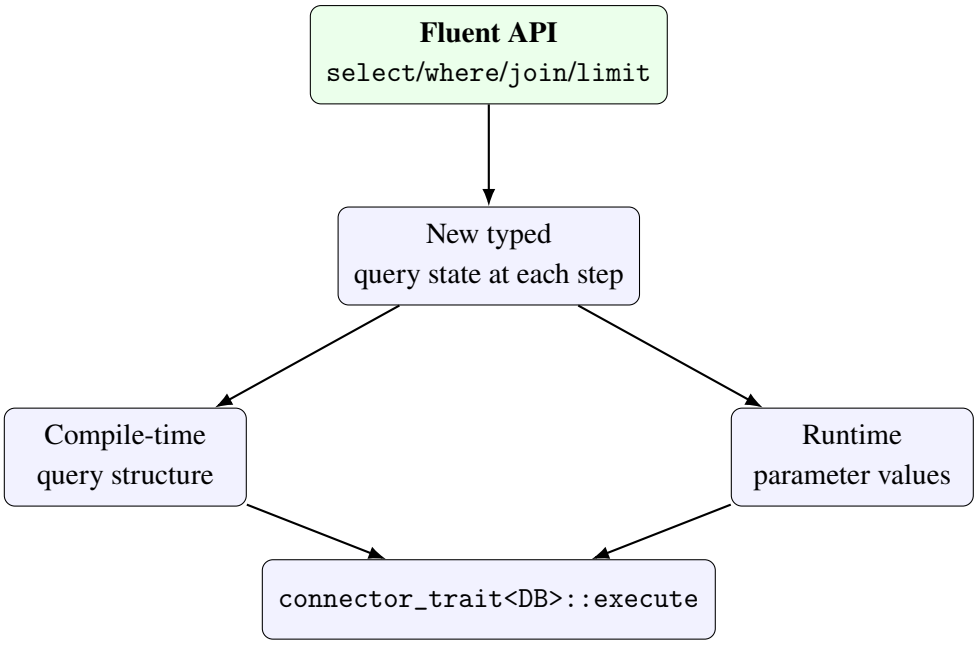

Figure 9. Interaction between query composition, compile-time structure, and runtime parameters

Figure 9 shows that the implementation operates in two modes at once: the structural form of the query is fixed in the type, while the concrete values are supplied separately to `execute(...)`. This is exactly what makes the coexistence of compile-time validation and runtime parameterisation possible.

5.8.5. Execution, results, and field access

At execution time, `db<DB>` forwards the query IR to `connector_trait<DB>::execute`. The result is returned as `orm::result<Row,FieldTuple>`. In this construction, `Row` is the materialised tuple type, while `FieldTuple` preserves the projection metadata for the selected fields. This solution allows both iteration over materialised rows and access through `get_field<&T::m>` or positional access through `get<I>`. The implementation therefore remains type-safe even after query execution.

The tests `SelectResultRowTypeIsTuple`, `SelectMultiFieldRowType`, `SelectGetByMemberPointer`, and `SelectGetByIndex` show that the result layer is not merely a container of anonymous rows, but a structured abstraction with type-traceable projection information. This matters for the thesis because it demonstrates that type discipline does not end with the query builder, but continues into result handling.

5.8.6. Prepared queries as a reusable execution artefact

`prepared_query<DB,Query>` extends the execution model. Instead of caching an SQL string, it stores a reference to the backend object and the query IR itself. This is a natural continuation of the compile-time approach: the library treats the query object as the primary artefact rather than as a transient step on the way to a textual result.

```
template <typename DB, typename Query>
class prepared_query
{
public:
    prepared_query(DB& conn, Query q) : conn_(conn), query_(std::move(q)) {}

    template <typename... Params>
    auto execute(Params&&... params) const
    {
        return connector_trait<DB>::execute(
            conn_, query_, std::forward<Params>(params)...);
    }

    [[nodiscard]] const Query& query() const noexcept { return query_; }
};
```

Listing 7: Excerpt from `prepared_query<DB,Query>`

Listing 7 reveals an important engineering choice: reuse is organised around the IR rather than around textual materialisation. This allows the same logical query to be executed repeatedly with different parameters without reconstructing the builder form. This behaviour is validated exactly by `PrepareReturnsExecutableObject`, `PrepareWithParamExecutesCorrectly`, `PrepareCanBeCalledMultipleTimesWithDifferentParams`, and `PrepareExposesQuery IR`.

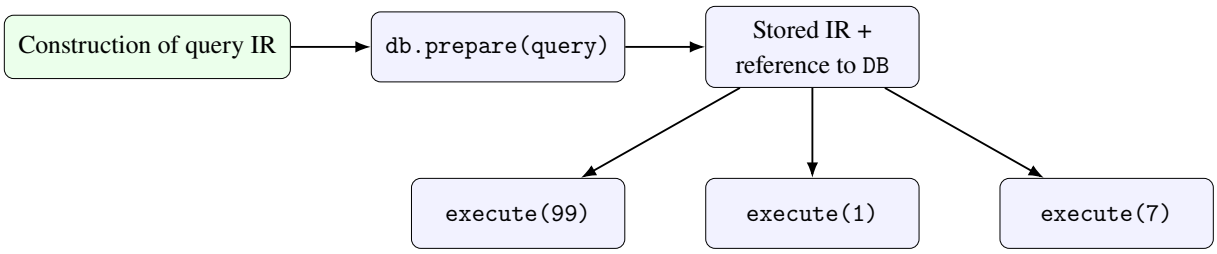

Figure 10. Lifecycle of `prepared_query`: one IR, multiple executions

5.8.7. Migration layer and DDL generation

The implementation of `migrate<DB>` compares the schema described by entity types against `live_schema`. Whenever a difference exists, DDL operations are created and forwarded to `connector_trait<DB>::ddl_for(...)`. This is a good example of clean separation of responsibilities: the diff logic is shared, whereas the concrete DDL syntax remains backend-specific.

```
[[nodiscard]] std::vector<ddl_op> diff(
    const live_schema& live,
    std::span<const entity_meta> entities) const
{
    std::vector<ddl_op> ops;

    for (const auto& entity : entities)
    {
        auto live_it = live.find(entity.table_name);
        if (live_it == live.end())
        {
            ops.push_back(create_table_op{entity.table_name, entity.columns});
            continue;
        }

        for (const auto& [col, type] : entity.columns)
        {
            if (live_it->second.find(col) == live_it->second.end())
                ops.push_back(add_column_op{entity.table_name, col, type});
        }
    }
    return ops;
}
```

Listing 8: Core of the migration diff pipeline

The distinction between a *minimally working connector* and a *full connector contract* becomes especially clear here. A backend may execute queries without supporting DDL generation. But if it is expected to participate in schema-aware tooling, its contract must be extended with `ddl_for(...)` hooks and consistent migration verification.

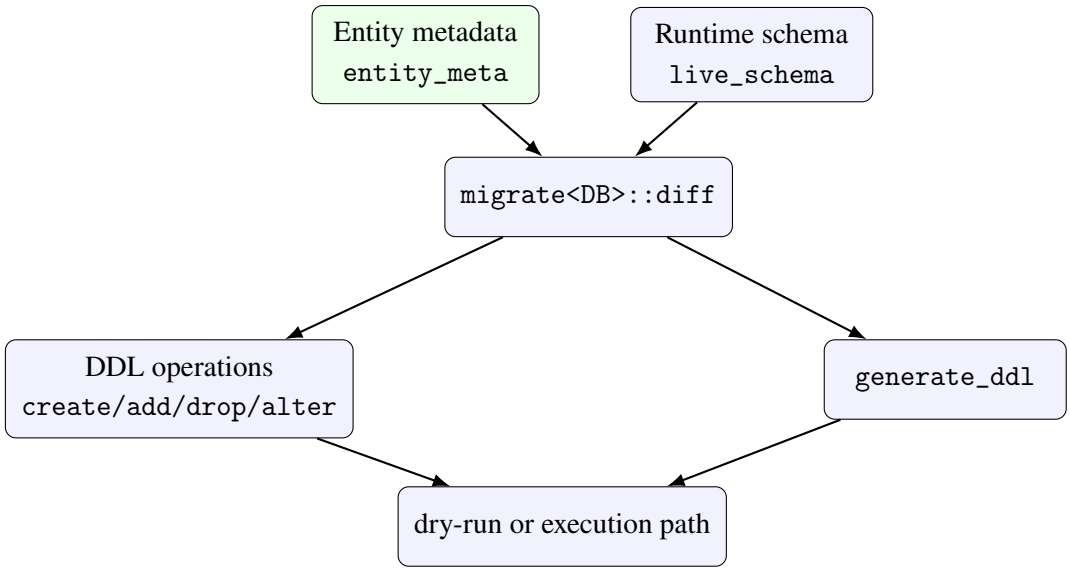

Figure 11. Flow of the schema-migration subsystem

The tests in `tests/unit/test_schema_migration.cpp` cover table creation, column addition and removal, dry-run completion codes, DDL generation, and the state-machine behaviour of `run()`. This is strong evidence that the migration layer is not merely a concept, but a genuinely traceable pipeline.

5.8.8. Thread safety and transaction helpers

The library contains a dedicated layer for `connection_pool<DB,N>`, `thread_local_db<DB>`, and `transaction_guard<DB>`. These components show that the implementation does not stop at one-off query execution, but also considers operational scenarios involving connection reuse, per-thread isolation, and rollback discipline. The important architectural choice is that access to these mechanisms is likewise capability-based. For example, `connection_pool` requires `supports_concurrent_execute`, whereas `transaction_guard` requires `supports_transactions`.

```
template <typename DB, std::size_t N>
class connection_pool
{
    static_assert(
        requires { typename connector_trait<DB>::supports_concurrent_execute; },
        "connection_pool requires "
        "connector_trait<DB>::supports_concurrent_execute. "
        "Add 'using supports_concurrent_execute = void;' "
        "to connector_trait<DB>.");

public:
    [[nodiscard]] connection_guard<DB> acquire();
```

```
};

template <typename DB>
class transaction_guard
{
    static_assert(
        requires { typename connector_trait<DB>::supports_transactions; },
        "transaction_guard requires "
        "connector_trait<DB>::supports_transactions. "
        "Add 'using supports_transactions = void;' "
        "to connector_trait<DB>.");
};
```

Listing 9: Excerpt from the thread-safety helper layer

In this way the compile-time contract is not restricted to query shape alone, but extends to the lifecycle policies of the connector. This is important for the thesis argument that the library is not merely a query builder, but infrastructure for systematic and safe work with backend resources.

Table 9. Principal subsystems and their test support

Subsystem	Principal files	Test support
Entity modelling	`ORM/entity/property.hpp`, `ORM/entity/relationship.hpp`	`tests/unit/test_property.cpp`, `tests/unit/test_relationship.cpp`
Query composition	`ORM/query/select.hpp` and the related query headers	`tests/unit/test_query.cpp`, `tests/integration/test_mockdb.cpp`
Prepared queries	`ORM/connector/prepared_query.hpp`	the `Prepare*` scenarios in `tests/integration/test_mockdb.cpp`
Schema migration	`ORM/db/migration/migration.hpp`	`tests/unit/test_schema_migration.cpp`
Thread safety	`ORM/db/connectors/ThreadSafety/thread_safety.hpp`	`tests/unit/test_thread_safety.cpp`

5.8.9. Implementation conclusion

The principal subsystems have direct support in both code and tests. `tests/unit/test_property.cpp` and `tests/unit/test_relationship.cpp` validate the entity descriptions. `tests/unit/test_query.cpp` proves that the query builder genuinely constructs correct compile-time structures. `tests/integration/test_mockdb.cpp` and `tests/integration/test_sqlite.cpp` validate execution, prepared-query scenarios, and result handling. `tests/unit/test_schema_migration.cpp` covers migration diff logic, while `tests/unit/test_thread_safety.cpp` checks the lifecycle layer for concurrent access.

Consequently, implementation of the principal subsystems is not merely a design intention, but an architectural decision backed by systematic verification. This is the most important conclusion of the present section: the compile-time ORM architecture in the repository does not remain at the level of abstraction, but materialises into working subsystems with clearly separated responsibilities and traceable repository evidence.

5.9. Architectural invariants

On the basis of the analysed layers, several invariants can be formulated. First, application code should never depend directly on driver APIs. Second, query logic is formulated in a typed IR rather than in backend-native strings. Third, extension toward a new backend proceeds through trait specialisation and capability declarations rather than through changes to the query API itself. Fourth, if an operation is inadmissible for a backend, this should become visible as early as possible — ideally at compile time.

These invariants make the architecture both engineering-sound and academically defensible. They allow every future extension to be judged not only by whether it “works”, but by whether it preserves the already formulated layers and boundaries.

5.10. Architectural conclusion

In summary, the architecture of the library separates the problem into three levels: logical model, typed query IR, and backend materialisation. This separation enables strict type discipline, formal extensibility, and controlled behaviour across distinct storage models. The next chapter shifts the focus to the second architectural axis — asynchronous execution — where it becomes clear how the same compile-time core is connected to coroutine lifecycle, thread-pool

offload, and native non-blocking integration.

VI. ASYNC EXECUTION ARCHITECTURE

Once the compile-time architectural model has been defined, the next question is how the library realises asynchronous execution as its second principal architectural axis. This chapter examines the main building blocks of the async layer and follows them from `Task<T>` and `co_await`-based composition to `ThreadPool`, `async_db<DB>`, `IoContext`, coroutine-aware connection pooling, and the integration with backend-specific execution paths.

The focus here is no longer on the query IR as a compile-time structure, but on the actual mechanics of suspend/resume execution. How is the synchronous connector API lifted into an asynchronous interface? Where is thread-pool offload the correct strategy and where is a native non-blocking API the more accurate model? How does the async layer guarantee correct use of connections and transactions without falling back to a callback-driven interface? The answers to these questions show how far the library is developed not only as a typed system, but also as an execution system.

6.1. Architectural position of the async layer

The asynchronous layer is placed above the already existing synchronous model of `db<DB>` and `connector_trait<DB>`. This is an important architectural choice. Rather than duplicating the query builder or introducing a second family of query types, the async subsystem accepts the compile-time constructed query IR as a finished input and is concerned only with *how* it is executed. In this way, the first architectural axis — typed description of data and operations — remains separated from the second axis — management of asynchronous lifecycle.

From that perspective, the async architecture has three principal responsibilities. First, it must provide a coroutine carrier for execution state and continuation. Second, it must provide a universal mechanism for lifting blocking connectors into a `Task`-based interface. Third, it must allow native non-blocking backends to participate in the same user syntax without the library simulating false uniformity where the drivers are in reality different.

6.2. `Task<T>` as the carrier of coroutine lifecycle

The basic user abstraction is `Task<T>`. Its role is not exhausted by "returning a future value". It is a container for the coroutine frame, the promise object, the continuation, and the completion policy. The implementation uses `initial_suspend()` and `final_suspend()` so

that the coroutine is lazy at the beginning and transfers control back to the continuation when it finishes. Consequently, `Task<T>` is at once an execution handle and a formal contract for how a result, an exception, and completion are transferred between coroutine contexts.

From an engineering standpoint, three properties matter especially. First, `operatorc o_await()` links the current continuation to the awaited task and enables natural nested `co_ await` composition. Second, `sync_wait()` provides a bridge to non-coroutine code, which is necessary both for unit tests and for boundary scenarios in which the application does not yet operate entirely coroutine-first. Third, `detach()` and `start_detached()` make controlled fire-and-forget behaviour possible, in which the coroutine frame destroys itself after `final_su spend()`.

This construction is not decorative. It determines when an async operation is lazy, when it begins execution, who is responsible for the frame lifetime, and how exceptions are propagated. Precisely for that reason, `Task<T>` stands at the centre of all other async components: `run_on_p ool()`, `async_db<DB>`, the `IoContext` awaitables, and the transaction helpers all work through the same coroutine contract.

6.3. `ThreadPool` and `run_on_pool()`: the universal execution path

The universal async strategy is thread-pool offload. `ThreadPool` maintains a bounded set of worker threads and a queue of tasks. Built on top of it is `run_on_pool()`, which accepts a callable, publishes it to the queue, suspends the current coroutine, and resumes it once the callable has completed. In that way, a synchronous connector API can be used through `co_awa it` without the user code falling back to manual promise/future plumbing.

The critical detail is that `run_on_pool()` is not merely a helper for "run something on another thread". Its awaiter stores the continuation, a possible exception, and the callable result. After execution on a worker thread, the continuation is resumed, and `await_resume()` returns the result or rethrows the error. The offload path therefore remains subject to the same coroutine rules as the native async path.

This universal model is especially important for backends that do not have a natural native non-blocking client interface or for which the library deliberately does not claim one. In those cases, the honest architectural choice is to isolate the blocking operation in `ThreadPool` while giving the user a uniform `Task`-based interface. Exactly this behaviour is validated by `RunOnPo olTest.*` and by `AsyncDbTest.AsyncSelectRunsOnPoolThread`, where it is demonstrated

that real execution moves away from the main thread.

6.4. `async_db<DB>` and capability-gated dispatch

The most important external async façade is `async_db<DB>`. It wraps `db<DB>` and preserves the familiar query syntax, but returns `Task<result<...>>`. The architectural significance of this wrapper is that it does not decide dynamically in runtime what to do; it uses compile-time capability gating instead. In `operator<<` there is an `ifconstexpr` branch over `has_capability<DB,cap::supports_async>`. If the connector declares native async support, `connector_trait<DB>::async_execute()` is called. Otherwise, the synchronous `execute()` path is wrapped in `run_on_pool()`.

```
template <typename Query>
[[nodiscard]] auto operator<<(Query q)
    -> Task<decltype(std::declval<db<DB>>() << std::declval<Query>())>
{
    using ResultType = decltype(std::declval<db<DB>>() << std::declval<Query>());

    if constexpr (has_capability<DB, cap::supports_async>)
    {
        co_return co_await connector_trait<DB>::async_execute(
            db_.connection(), std::move(q));
    }
    else
    {
        co_return co_await run_on_pool(
            *pool_, [this, q = std::move(q)]() mutable -> ResultType {
                return db_ << std::move(q);
            });
    }
}
```

Listing 10: The central capability-gated branch in `async_db<DB>`

Listing 10 is central to the entire async architecture. It shows that the library does not use a runtime registry or string-based dispatch, but a compile-time check over connector capabilities. In this way, the user API remains uniform while the internal execution path remains faithful to the real abilities of a given backend.

The helper method `async_execute(Query,Params...)` plays a similar bridging role for imperative scenarios with explicit parameters, while `sync_handle()` provides a controlled escape hatch to the synchronous interface when such a boundary is needed. Consequently, `async_db<DB>` is not a parallel library, but an adaptation layer between the compile-time ORM core

and the coroutine execution model.

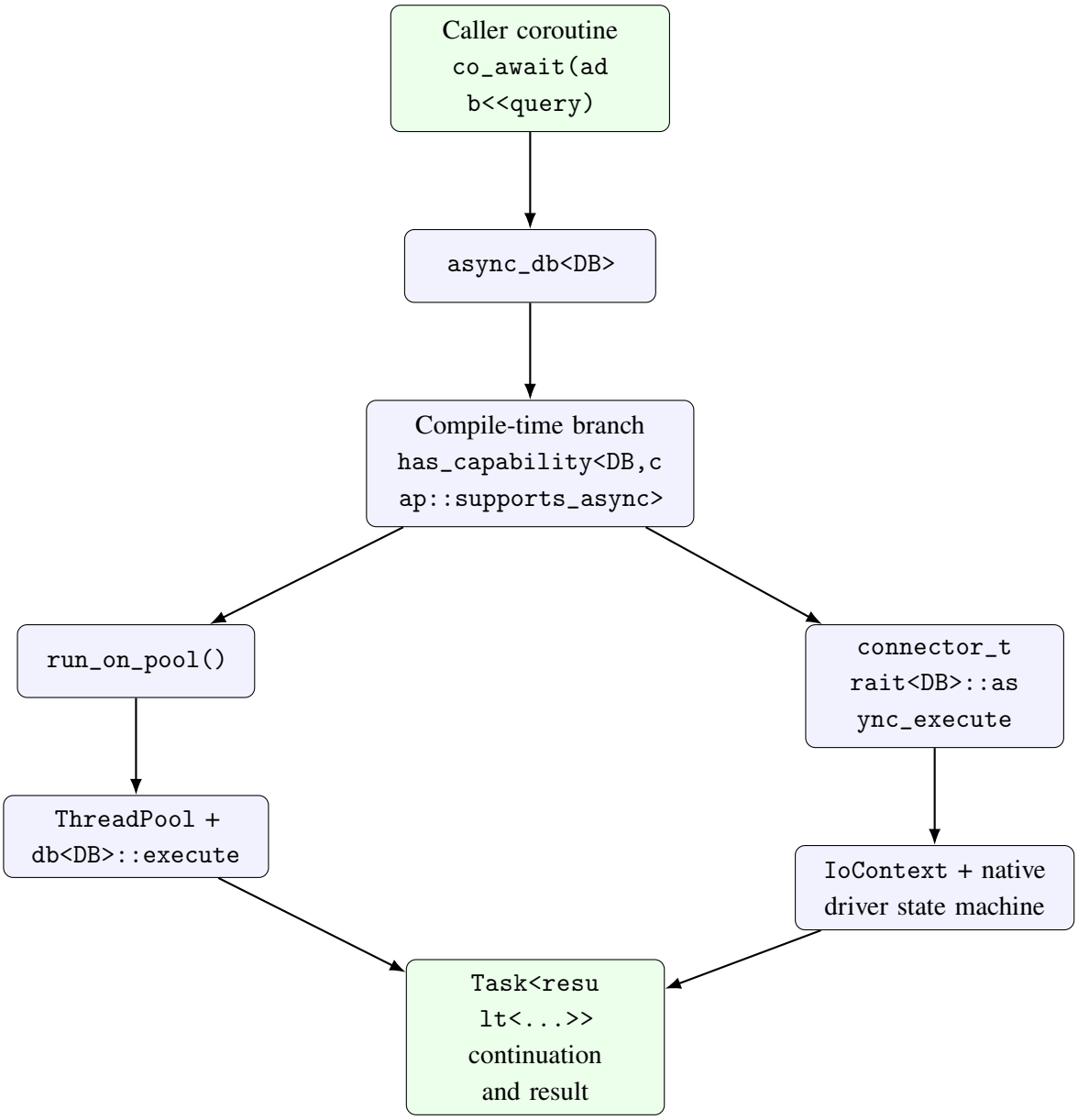

Figure 12. The two execution paths of `async_db<DB>`: universal offload and native async integration

Figure 12 shows the most important architectural decision in the async layer. The external interface remains uniform, but the internal execution path is capability-gated. That is why the library can simultaneously offer a convenient coroutine API and avoid the false claim that all drivers are naturally non-blocking.

6.5. `IoContext` and native non-blocking integration

For drivers with a genuine native async interface, the library does not restrict itself to thread-pool offload. Here `IoContext` becomes active — an event-loop abstraction that provides `run()`, `run_one()`, `stop()`, `post()`, `schedule()`, and, most importantly, `watch_readable(fd)` and `watch_writable(fd)`. These awaitables do not perform input/output (I/O) themselves. They merely suspend the coroutine and resume it once the file descriptor becomes ready for reading or writing.

This separation between readiness and the I/O action itself is decisive. `IoContext` does not attempt to hide the driver, but provides a coroutine-friendly reactor interface on top of which the native client library can operate. In the implementation of `AsyncPostgreSQLDB`, this is especially clear. Asynchronous connection establishment uses `PQconnectPoll()`, and the coroutine successively awaits `watch_writable()` or `watch_readable()` according to the state of the `libpq` state machine. During query execution, `PQsendQueryParams()`, `PQflush()`, `PQisBusy()`, and `PQconsumeInput()` are combined with the same `IoContext` mechanism.

The native async path is therefore not a "faster thread pool", but a qualitatively different execution model. The worker thread does not block while waiting for network readiness; the coroutine is suspended, and resumption is governed by event-loop observation. That is precisely the architectural model that the next chapter develops per backend: PostgreSQL, MySQL, Redis, and Cassandra can offer native async integration in different operational forms, whereas other drivers remain honestly described as blocking and are therefore served by the universal offload path.

6.6. Coroutine-aware connection pooling and transaction helpers

The async architecture does not end with single-query execution. A real application must manage connections and transactions under concurrent access. That is precisely why the library provides `async_connection_pool<DB,N>`, `async_connection_guard<DB>`, `async_transaction_guard<DB>`, and the factory coroutine `async_begin_transaction()`.

`async_connection_pool<DB,N>` is a coroutine-aware continuation of the synchronous pooling layer. Its `acquire()` method returns `Task<async_connection_guard<DB>>`. Internally it uses `run_on_pool()` and a condition variable to wait for a connection slot whose state is `Idle`, without exposing the calling coroutine to a blocking interface. The obtained guard

follows the Resource Acquisition Is Initialization (RAII) idiom: upon destruction it returns the connection state to `Idle` and notifies any waiters.

In turn, `async_connection_guard<DB>::get()` does not return a raw connection, but a new `async_db<DB>` handle. This is an important architectural detail. Even after acquiring a concrete connection resource, the user remains in the same coroutine execution model and does not fall back to a low-level API.

The same logic is visible for transactions. `async_begin_transaction()` first offloads `BEGIN` to the pool and then returns `async_transaction_guard<DB>`. The methods `commit()` and `rollback()` are `Task<void>` operations, while the destructor of the guard performs a protective `ROLLBACK` if the transaction has not been finalised explicitly. This design is both convenient and conservative: the success path is coroutine-friendly, while the cleanup path remains deterministic.

These properties are exactly what is validated by `AsyncConnectionPoolTest.ConcurrentAcquires`, `AsyncTransactionTest.BeginAndCommit`, `AsyncTransactionTest.RollbackOnDestruction`, and `AsyncTransactionTest.ExplicitRollback`. Consequently, the pooling and transaction helpers are not peripheral conveniences, but an important part of the overall execution architecture.

Table 10. Principal async subsystems and their code and test evidence

Subsystem	Architectural role	Main files and tests
`Task<T>`	coroutine lifecycle, continuation, `sync_wait()`, and detach semantics	`lib/include/ORM/async/task.hpp`, `tests/unit/test_task.cpp`
`ThreadPool` and `run_on_pool()`	universal offload path for blocking operations	`lib/include/ORM/async/thread_pool.hpp`, `tests/unit/test_thread_pool.cpp`
`async_db<DB>`	capability-gated async dispatch over `db<DB>`	`lib/include/ORM/connector/async_db.hpp`, `tests/unit/test_async_connector.cpp`
`IoContext` and native async connectors	reactor-style fd readiness and native non-blocking integration	`lib/include/ORM/async/io_context.hpp`, `lib/include/ORM/db/connectors/PostgreSQLDB/postgresql_async.hpp`, `tests/unit/test_io_context.cpp`
Pooling and transactions	coroutine-aware connection ownership and transactional safety	`lib/include/ORM/db/connectors/ThreadSafety/async_thread_safety.hpp`, `tests/unit/test_async_connector.cpp`

6.7. Architectural conclusion on the async layer

The asynchronous architecture of the library is not merely an added coroutine wrapper around the synchronous ORM core. It is a distinct layer with clearly separated responsibilities: `Task<T>` models coroutine lifecycle, `run_on_pool()` provides a universal offload mechanism, `async_db<DB>` realises capability-gated dispatch, `IoContext` provides native readiness integration, and the pooling and transaction helpers extend the model toward practically relevant concurrent scenarios.

Consequently, the second architectural axis of the thesis does not reduce to “having an async API”. It shows how the compile-time formalised access model can be executed through two honestly differentiated asynchronous execution paths — universal and native — without the library losing type discipline, lifecycle safety, or connector transparency. It is precisely this layer that makes the subsequent analysis of backend execution models possible.

VII. BACKEND EXECUTION MODELS AND NATIVE CLIENT LIBRARIES

This chapter examines not only how one and the same C++ abstraction is materialised across different backends, but also how it is executed through the concrete native client libraries. The analysis has two goals. First, it demonstrates that the library is not restricted to a purely SQL interpretation. Second, it clarifies where asynchronous integration can follow a native non-blocking model and where the honest architectural choice remains thread-pool offload over a synchronous driver.

After the theoretical framework of Chapter III and the explanation of the async layer in Chapter VI, the question is no longer whether the different storage models are equivalent. Instead, the question is whether one and the same logical ORM form can be adapted to them in a controlled way, without the library promising execution semantics that a given backend cannot naturally provide. It is precisely here that the value of the capability-based connector contract and of the distinction between a universal async interface and a backend-specific implementation becomes visible.

7.1. Methodology of the analysis

Each scenario follows the same sequence: theoretical basis, logical mapping from the C++ model, implementation in `connector_trait<DB>`, limitations, and verification through tests. This makes comparison between backends possible. The aim is not to claim that all systems are equivalent, but that one and the same logical ORM abstraction can be adapted in a controlled fashion to distinct storage models.

Table 11. Common analytical frame for each backend scenario

Criterion	What is evaluated
Theoretical basis	to which storage model from Chapter III the backend belongs
Logical mapping	how `property`, `field`, `Rule`, and `table_name_trait` are interpreted in the backend
Capability profile	which ORM operations are explicitly admitted and which must be rejected
Execution behaviour	how queries, parameters, and result projections are rendered
Test evidence	which unit and integration tests validate the contract in the repository

This analytical frame matters because it keeps the comparison honest. SQL backends are not privileged merely because they expose a richer syntax, and NoSQL backends are not judged as "incomplete" merely because they reject operations that are inappropriate for their storage model.

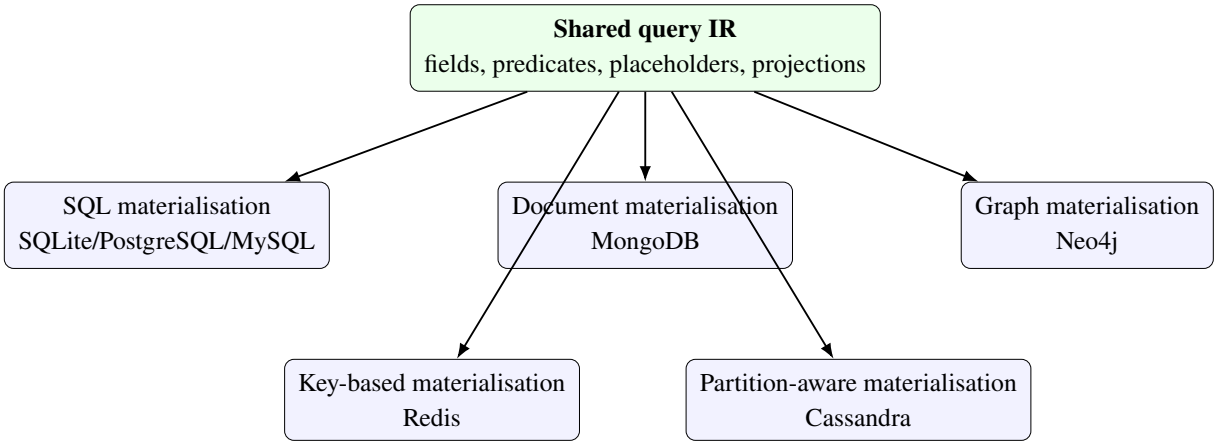

Figure 13. One shared query IR and multiple backend-specific materialisation paths

Figure 13 expresses the core claim of the chapter: there are not multiple unrelated ORM libraries in the repository, but one logical library with multiple materialisation paths. This is the strong argument in favour of the connector architecture — backend diversity is managed through a shared IR and an explicit contract rather than by duplicating APIs.

7.2. Relational baseline scenario: SQLite

This scenario rests on the relational model discussed in Chapter III. `SQLiteDB` is the most natural reference backend for the library because it combines real SQL semantics, a compact

operational context, and direct integration through the `sqlite3` API. In this backend, `property` materialises as a column, `table_name_trait` as a table name, and `Rule` trees as `WHERE` expressions.

The connector declares `supports_joins`, `supports_transactions`, and `supports_aggregation`, which makes it the closest to classical ORM expectations. It is precisely here that prepared queries, binding logic, `find_one()` behaviour, and result hydration into typed tuples are validated. The integration tests in `tests/integration/test_sqlite.cpp` demonstrate not only correctness of `SELECT`, `INSERT`, `UPDATE`, and `DELETE`, but also behaviour under a transaction-capable and join-capable connector profile.

From an academic point of view, SQLite plays the role of relational baseline. If a given ORM form is problematic already here, it is unlikely to generalise well toward more exotic backends. For that reason, the present work uses SQLite as a reference point rather than merely as one example among many.

7.3. Relational portability: PostgreSQL and MySQL

The theoretical basis here is again the relational model, but the architectural interest is different: to what extent one and the same query IR remains valid across different SQL dialects and different parameter-binding rules. `PostgreSQLDB` and `PostgreSQLLiveDB` support `supports_joins`, `supports_transactions`, and `supports_aggregation`, while the tests in `tests/unit/test_postgresql_connector.cpp` and `tests/integration/test_postgresql_live.cpp` show real execution against a live database.

`MySQLDB` and `MySQLLiveDB` also declare `supports_joins` and `supports_transactions`, but here the difference in binding model becomes important. While PostgreSQL can naturally address parameters through `$1`, `$2`, ..., MySQL relies on positional `?` placeholders. This does not break the shared IR, but it demonstrates that logical portability does not imply byte-for-byte identical backend materialisation.

These SQL family scenarios therefore defend a stronger thesis than “the library works with SQL”: the query IR is general enough for more than one dialect, yet honest enough to leave low-level differences within the connector layer.

7.4. Document scenario: MongoDB

This scenario is based on the document model from the theoretical chapter. In MongoDB, `property` is interpreted as a document field, while query predicate trees are translated into BSON-like filters. `select` expressions materialise as projections, and the notion of table in ORM terms becomes a collection. This is precisely where the practical value of a storage-agnostic query IR becomes visible: the user does not describe `\$eq`, `\$and`, or projection directly, yet the connector can derive them from the same logical form.

An important detail is that `connector_trait<MongoDB>` does not declare SQL-style capability tags. This is not an omission, but a correct description of the connector contract. There is no attempt to claim that the Mongo backend is join-equivalent to relational connectors. Instead, it provides a document-native interpretation of the query IR. The unit and integration tests in `tests/unit/test_mongodb_connector.cpp` and `tests/integration/test_mongodb_live.cpp` validate this adaptation against a real driver layer.

The architectural value of this scenario is that it demonstrates an important restriction: a library may be multi-backend without claiming that all backends support the same operations. It is exactly this sort of controlled partiality that makes the design scientifically defensible.

7.5. Key-value scenario: Redis

This scenario rests on key-value theory. In Redis, one and the same entity model can be materialised in two principal ways: as `SET/GET` for single-field structures or as `HSET/HGETALL` for multi-field structures. `table_name()` participates in generation of the key prefix, while access is concentrated around primary-key equality.

Here it becomes very clear that the shared query IR does not imply identical expressiveness. `RedisDB` deliberately does not declare `supports_joins`, `supports_transactions`, or `supports_aggregation`. This restriction is part of the correct contract rather than a weakness of the architecture. The unit and live tests in `tests/unit/test_redis_connector.cpp` and `tests/integration/test_redis_live.cpp` prove that the library correctly materialises key-based access without promising unsupported operations.

This scenario is especially valuable for the thesis because it forces the architecture to distinguish clearly between the *logical data model* and the *admissible access model*. That distinction is easy to forget in SQL-first libraries, but here it is elevated into a central principle.

7.6. Graph scenario: Neo4j

The scenario for Neo4j is based on the graph model. The entity type materialises as a label and a set of properties, while the query IR is translated into `MATCH`, `RETURN`, and `WHERE` fragments. This is an important architectural test because graph-query semantics are among the furthest from the classical SQL model. Nevertheless, the logical elements — fields, predicates, filtering, and result projections — remain recognisable.

`Neo4jDB` and `Neo4jLiveDB` declare `supports_transactions`, but not SQL join/aggregation capabilities. This is meaningful: graph traversal is not the same as SQL join, even though both connect data at a logical level. The unit and integration tests in `tests/unit/test_neo4j_connector.cpp` and `tests/integration/test_neo4j_live.cpp` show that the abstraction can adapt one and the same logical model toward a graph backend if the connector takes responsibility for correct materialisation.

From the standpoint of the thesis, this is one of the strongest demonstrations that the query IR is semantic rather than syntactic. If the IR were merely under-specified SQL, the Neo4j scenario would collapse. The fact that a working graph materialisation is possible supports the central architectural claim.

7.7. Wide-column scenario: Cassandra

This scenario is grounded in wide-column theory. Although Cassandra offers a CQL syntax reminiscent of SQL, the actual access model is governed by partition keys and the distributed organisation of data. The library therefore cannot treat Cassandra as an ordinary SQL backend. The mere existence of a `WHERE` clause is not enough; it must also remain compatible with partition logic.

`CassandraDB` and `CassandraLiveDB` declare `supports_transactions`, but not `supports_joins`. This is highly revealing. On the one hand, the backend remains part of the same ORM library. On the other hand, the contract categorically refuses operations that would be misleading or expensive in a wide-column environment. The unit and live tests in `tests/unit/test_cassandra_connector.cpp` and `tests/integration/test_cassandra_live.cpp` provide concrete verification of this stricter operational profile.

For that reason, Cassandra is the best example that the extensibility of the library is not based on false equivalence, but on formally described limits of admissible operations.

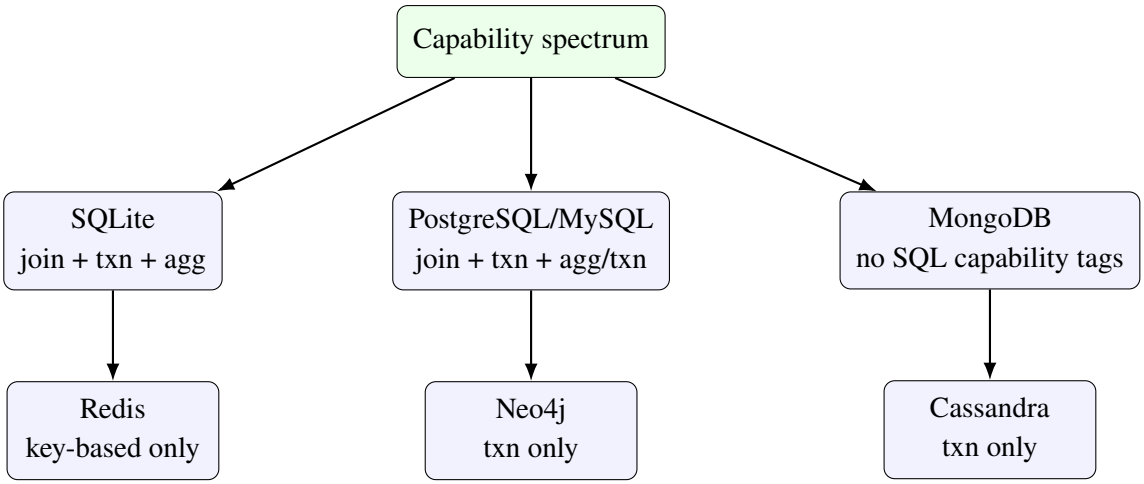

Figure 14. Comparative capability profile of the backends in the repository

Figure 14 synthesises one of the most important observations of the chapter: the library does not impose the same capability profile on every backend. On the contrary, the differences are used as first-class architectural information.

7.8. Synthesised capability matrix

To keep the comparison not only qualitative but also tabularly traceable, Figure 14 can be reduced to a compact capability matrix. It should not be read as a universal truth about the database systems themselves, but as a description of the concrete architectural position of the library toward them in the examined repository revision.

Table 12. Capability profiles of the principal backends according to the available tests and connector specialisations

Backend	Joins	Transac-tions	Aggre-gation	Upsert	Bulk insert	Short comment
SQLite	present	present	present	limited	limited	the closest full relational reference path in the repository
PostgreSQL	present	present	present	specific	possible	strong relational contract with live execution evidence
MySQL	present	present	present	specific	possible	the operational path is available, but binding and dialect details remain connector-specific
MongoDB	absent	absent	absent	analog.	not central	strong document model without an SQL-style capability profile
Redis	absent	absent	absent	analog.	limited	key-value semantics with a minimalist but honest contract profile
Neo4j	analog.	present	absent	not central	not central	graph backend with transaction focus rather than an SQL join/aggregation layer
Cassandra	absent	present	absent	not central	possible	wide-column backend with a restricted but formally defined capability set

Table 12 makes it easier to see where the library comes closest to classical ORM behaviour and where it deliberately works with a partial contract. Relational backends concentrate the richer capability profile, whereas document, graph, and key-value variants are

evaluated through other criteria. It is exactly this controlled non-uniformity that makes the multi-backend design engineering-honest.

7.9. Comparison between test evidence and backend contract

Table 13. Backends, capability profile, and available test evidence

Backend	Capability profile	Unit/contract test	Integration/live test
SQLite	joins, transactions, aggregation	capability asserts in `tests/integration/test_sqlite.cpp`	`tests/integration/test_sqlite.cpp`
PostgreSQL	joins, transactions, aggregation	`tests/unit/test_postgresql_connector.cpp`	`tests/integration/test_postgresql_live.cpp`
MySQL	joins, transactions, aggregation	`tests/unit/test_mysql_connector.cpp`	`tests/integration/test_mysql_live.cpp`
MongoDB	no SQL capability tags	`tests/unit/test_mongodb_connector.cpp`	`tests/integration/test_mongodb_live.cpp`
Redis	no joins/ transactions/ aggregation	`tests/unit/test_redis_connector.cpp`	`tests/integration/test_redis_live.cpp`
Neo4j	transactions	`tests/unit/test_neo4j_connector.cpp`	`tests/integration/test_neo4j_live.cpp`
Cassandra	transactions	`tests/unit/test_cassandra_connector.cpp`	`tests/integration/test_cassandra_live.cpp`

Table 13 is important because it translates the comparison from the level of narrative analysis to the level of verifiable repository evidence. For every backend there is at least one explicit contract-level or live-level correlate. This reduces the risk that the chapter becomes a declarative survey without technical support.

7.10. Comparative synthesis

The comparison across backends shows that the library succeeds in preserving a shared logical model at the level of entity types, fields, placeholders, and query IR. Differences appear at the level of capabilities and physical data organisation. This is the central conclusion of the chapter: a successful multi-backend ORM architecture does not erase differences between storage models, but makes them controllable through a formal connector contract.

Table 14. Comparison of backend materialisation

Backend	Primary structure	Logical interpretation	Representative limitation
SQLite	table	full SQL ORM interpretation	standard relational discipline and rich query expressiveness
PostgreSQL/MySQL	table	portable SQL IR	different binding models and dialect details
MongoDB	document	filter/projection interpretation	only partial equivalence with SQL operations
Redis	key/value, hash	key-based entity access	lookup mostly by primary key and no join semantics
Neo4j	node/properties	graph-pattern interpretation	graph-specific query semantics rather than SQL join behaviour
Cassandra	wide-column row	CQL with key constraints	partition-driven access and restricted predicate profile

As a result, the multi-backend design of the library can be defended with a more precise formulation. The system does not promise a “universal database through one API”, but rather a *unified logical layer with explicit backend boundaries*. This formulation is both

engineering-honest and scientifically grounded.

VIII. VERIFICATION AND EMPIRICAL EVALUATION

This chapter brings together the two forms of evidence on which the present work relies. On the one hand stand the compile-time constraints, unit tests, integration tests, and live-backend checks through which the correctness of the architecture is validated. On the other hand stands the empirical evaluation of the asynchronous layer, whose goal is to show when coroutine-based execution yields a real practical benefit and when its cost remains comparable to or higher than the synchronous alternative.

The important methodological principle is that these two forms of evidence are not treated as interchangeable. Test-based verification answers whether the library is correct with respect to its own contracts and backend limitations. Benchmark evaluation answers a different question: does the asynchronous execution model deliver a measurable benefit under workloads in which the waiting phase dominates local computation. Only the joint presence of both layers makes the architectural claims of the work sufficiently convincing.

8.1. Verification strategy

The verification strategy follows the architecture of the library itself. The compile-time layer is checked through concepts, capability restrictions, and typed query-IR construction. The runtime layer is checked through unit tests for rendering, parameter binding, thread safety, migrations, coroutine infrastructure, and asynchronous access. The outermost layer is validated through integration tests against real backends so that mock correctness is not confused with actual driver behaviour.

In the current repository revision, the full automated suite contains 271 tests when executed through `ctest`. That number is not in itself a scientific result, but it is important as an indicator that the mechanisms discussed in the preceding chapters are not left at the level of a conceptual scheme. They are covered by systematic checks across layers and backend families.

Table 15. Principal sources of verification evidence

Source	What it validates	Representative artefacts
Compile-time constraints and query IR	type correctness of queries, capability-based restriction, and connector contracts	`tests/unit/test_query.cpp`, `tests/unit/test_postgresql_connector.cpp`, `tests/unit/test_mongodb_connector.cpp`
Asynchronous infrastructure	`Task<T>`, `ThreadPool`, `IoContext`, `async_db<DB>`, coroutine-aware pooling, and transactions	`tests/unit/test_task.cpp`, `tests/unit/test_thread_pool.cpp`, `tests/unit/test_io_context.cpp`, `tests/unit/test_async_connector.cpp`
Migrations and thread safety	schema-diff logic, rollback semantics, connection guards, and concurrency safety	`tests/unit/test_schema_migration.cpp`, `tests/unit/test_thread_safety.cpp`
Live backend integration	behaviour across real SQLite, PostgreSQL, MySQL, MongoDB, Redis, Neo4j, and Cassandra environments	`tests/integration/test_sqlite.cpp`, `tests/integration/test_postgresql_live.cpp`, `tests/integration/test_mongodb_live.cpp`, `tests/integration/test_redis_live.cpp`
Empirical evaluation	throughput under simulated waiting and isolated coroutine-overhead measurements	`benchmarks/bench_async.cpp`

Table 15 shows that the evidence base is heterogeneous by necessity. A compile-time ORM library cannot be defended through throughput numbers alone, just as an asynchronous architecture cannot be defended through concept checks alone. Both kinds of evidence are required.

8.2. Catalogue of unit and integration evidence

To prevent the verification strategy from remaining at the level of a general framework only, it must also be unfolded as a concrete catalogue of repository artefacts. This is especially

important for the present work because the architectural claims are distributed across the entity layer, the query IR, the connector contract, the asynchronous infrastructure, and live-backend checks. The following tables do exactly that: they show where the principal evidence is located in the repository and what role each test group plays.

Table 16. Principal unit tests for the generic, core, and asynchronous subsystems

File	Subsystem	What it validates
`test_types.cpp`	basic type aliases	correctness of the public aliases and basic utility dependencies
`test_property.cpp`	entity property layer	compile-time model of scalar fields and type-trait behaviour
`test_relationship.cpp`	entity relationship layer	inference logic for relations and storage-strategy behaviour
`test_query.cpp`	query IR and fluent API	correctness of `select`, `insert`, `update`, `delete q`, and the derived query structures
`test_schema_migration.cpp`	migration layer	DDL generation, diff logic, and state-machine behaviour
`test_thread_safety.cpp`	concurrency helpers	behaviour of the connection pool, thread-local access, and transaction-guard mechanism
`test_task.cpp`	coroutine task abstraction	lifecycle of `Task<T>` and basic await/coroutine semantics
`test_thread_pool.cpp`	worker scheduling	posting, scheduling, and worker-resource coordination
`test_io_context.cpp`	asynchronous orchestration	event-loop and continuation plumbing for the asynchronous execution layer
`test_async_connector.cpp`	asynchronous connector facade	bridge logic between `async_db<DB>` and coroutine-aware connector hooks
`test_wire_protocol.cpp`	experimental execution layer	low-level compile-time and wire-oriented mechanisms
`test_cpp26_reflection.cpp`	future language integration	early path for reflection-based automation

Table 16 matters because it shows that the unit layer is not limited to query-parser-like checks. It simultaneously covers the logical model, execution mechanics, migrations, and the coroutine-aware infrastructure on which the asynchronous architecture depends.

Table 17. Backend-oriented unit tests and their contract focus

File	Backend focus	Type of evidence
`test_postgresql_connector.cpp`	`PostgreSQLDB`	`is_connector` compliance, capability asserts, and SQL parameter rendering
`test_mysql_connector.cpp`	`MySQLDB`	joins/transactions contract, dialect-specific rendering, and binding expectations
`test_mongodb_connector.cpp`	`MongoDB`	negative capability validation and BSON-like filter rendering
`test_redis_connector.cpp`	`RedisDB`	absence of joins/transactions/aggregation and key-value command materialisation
`test_cassandra_connector.cpp`	`CassandraDB`	capability restrictions and CQL-rendering correlates
`test_neo4j_connector.cpp`	`Neo4jDB`	transaction capability and graph-oriented query materialisation

These backend-oriented unit tests are critical because they prove not only positive capabilities, but also honestly declared limitations. In a multi-backend library, negative verification — that a backend *does not* support an operation — is just as important as positive verification.

Table 18. Catalogue of integration and live tests

File	Execution mode	What it demonstrates
`test_mockdb.cpp`	always-available integration baseline	query composition, parameter binding, prepared queries, and CRUD scenarios without an external backend
`test_sqlite.cpp`	local integration test	real relational execution path with capability asserts, joins, transactions, and result hydration
`test_postgresql_live.cpp`	conditional live test	real connection, table lifecycle, and CRUD execution against PostgreSQL
`test_mysql_live.cpp`	conditional live test	real MySQL driver path, insert/update/delete, and select filtering
`test_mongodb_live.cpp`	conditional live test	execution of document-oriented scenarios against a live backend
`test_redis_live.cpp`	conditional live test	key-value and hash-command materialisation in a Redis environment
`test_cassandra_live.cpp`	conditional live test	wide-column execution and backend-specific CQL scenarios
`test_neo4j_live.cpp`	conditional live test	graph-query execution through a controlled Docker environment

Table 18 shows that the repository-backed evidence is layered. Not all backends have the same operational cost for live verification, but the repository contains a clear strategy for how and when such tests are activated. This matters especially for the thesis because it avoids confusing mock correctness with real driver-backed behaviour.

8.3. Maturity map according to evidence support

The capability profile by itself is not sufficient if evidence for actual behaviour is missing. It is therefore useful to examine the backends also as a maturity ladder determined not only by supported operations, but also by available test support. Here maturity means a combination of contract correctness, execution evidence, and operational realism.

Table 19. Backend maturity map according to the available evidence base

Backend	Maturity level	Principal evidence base	Interpretation
MockDB	level 3	test_mockdb.cpp	backend-independent execution baseline for the query contract and prepared-query mechanics
SQLite	level 3 to 4	test_sqlite.cpp	local real execution path with rich query evidence and capability asserts
PostgreSQL	level 3	test_postgresql_connector.cpp, test_postgresql_live.cpp	strong contract plus conditional live-backend verification
MySQL	level 3	test_mysql_connector.cpp, test_mysql_live.cpp	real operational path with a relational profile and dialect-specific binding
MongoDB	level 3	test_mongodb_connector.cpp, test_mongodb_live.cpp	document-oriented execution evidence without SQL-style capabilities
Redis	level 3	test_redis_connector.cpp, test_redis_live.cpp	key-value execution evidence with a restricted but honest contract profile
Neo4j	level 3	test_neo4j_connector.cpp, test_neo4j_live.cpp	graph execution path with Docker-gated live verification
Cassandra	level 3	test_cassandra_connector.cpp, test_cassandra_live.cpp	wide-column path focused on formal suitability and live coverage

The map in Table 19 is important because it prevents an overly binary reading of backend status. A connector is not simply present or absent; it may be contract-correct, execution-validated, partially live-verified, or schema-aware rich. It is exactly this more nuanced reading that makes the verification chapter scientifically stronger and closer to the real state of the repository.

8.4. Benchmark methodology

The empirical evaluation is implemented through Google Benchmark [5] in `benchmarks/bench_async.cpp`. Its purpose is not to simulate the full lifecycle of a concrete database, but to isolate the architectural question of whether coroutine-based execution frees worker threads more effectively than synchronous blocking when a short local CPU phase is separated by a waiting interval.

The benchmark suite is divided into two groups. The first group measures the throughput of whole batches of tasks. The synchronous baseline performs CPU work and then blocks the worker thread for a predefined interval. The asynchronous variant uses the `SimulatedAsyncIO` awaitable, which suspends the coroutine, schedules a delayed resumption through a timer queue, and posts the continuation back to `ThreadPool`. The second group measures isolated overheads: creation and waiting of `Task<int>`, the `ThreadPool::post()` round-trip, and nested `co_await` chains.

Methodologically, it is important that the throughput benchmarks use wall-clock time rather than pure CPU time. The reason is explicitly stated in the benchmark source itself: on macOS, calibration by CPU time may behave misleadingly for coroutine scenarios with a waiting phase because a suspended task consumes almost no CPU. The comparison is therefore performed through `UseRealTime()` and a fixed number of iterations.

This chapter uses the release-mode results from the local `build-rel` build tree. They should not be interpreted as absolute production numbers for all possible machines and drivers. Their role is comparative: all variants are executed on the same machine and with the same parameter grid so that the relative effect of the different execution models can be isolated.

Table 20. Benchmark families and their purpose

Benchmark family	What it measures	Principal parameters
`BM_SyncThroughput`	throughput of the blocking baseline, in which the worker thread remains occupied during the waiting phase	number of threads, number of tasks, waiting latency, CPU iterations
`BM_AsyncThroughput`	throughput of the coroutine variant with suspend/resume and delayed resumption through a timer queue	number of threads, number of tasks, waiting latency, CPU iterations
`BM_AsyncYieldThroughput`	cost of coroutine interleave without artificial waiting latency	number of threads, number of tasks, CPU iterations
`BM_TaskCreateAndWait`	minimal cost of creating and completing a trivial task	single coroutine without external scheduling
`BM_ThreadPoolPost`	cost of `post()` and the promise/future round-trip to a worker thread	size of the thread pool
`BM_NestedCoAwait`	overhead of nested `co_await` continuation at different depths	depth of the coroutine chain

8.5. Principal throughput results

Table 21 summarises representative release-mode results for the most informative configurations. It shows direct pairs for which the synchronous and asynchronous variants were measured under the same parameters.

Table 21. Representative release-mode throughput results for synchronous and asynchronous execution

Threads	Tasks	Waiting latency	Synchronous throughput	Asynchronous throughput	Speedup
4	100	10 μs	194.6k ops/s	658.5k ops/s	3.38×
4	100	100 μs	29.8k ops/s	458.2k ops/s	15.35×
4	100	1000 μs	3.22k ops/s	75.28k ops/s	23.36×
4	1000	100 μs	30.08k ops/s	670.5k ops/s	22.29×
8	100	100 μs	57.13k ops/s	258.5k ops/s	4.53×

The most important conclusion is that the advantage of the coroutine model grows not when local CPU work grows, but when the waiting phase begins to dominate it. With 4 worker threads and 100 tasks, increasing the waiting latency from 10 to 1000 microseconds (μs) raises the relative advantage of the asynchronous variant from 3.38× to 23.36×. This is exactly the expected behaviour: in the synchronous baseline, threads are exhausted by blocking, whereas in the coroutine model they are released for other work.

A second important conclusion is that the effect does not disappear for larger batches. With 4 threads, 1000 tasks, and 100 μs waiting latency, the asynchronous variant reaches 670.5k operations per second versus 30.08k for the synchronous baseline. This shows that the architectural advantage is not limited to a small demonstration case, but persists when competition for worker resources increases.

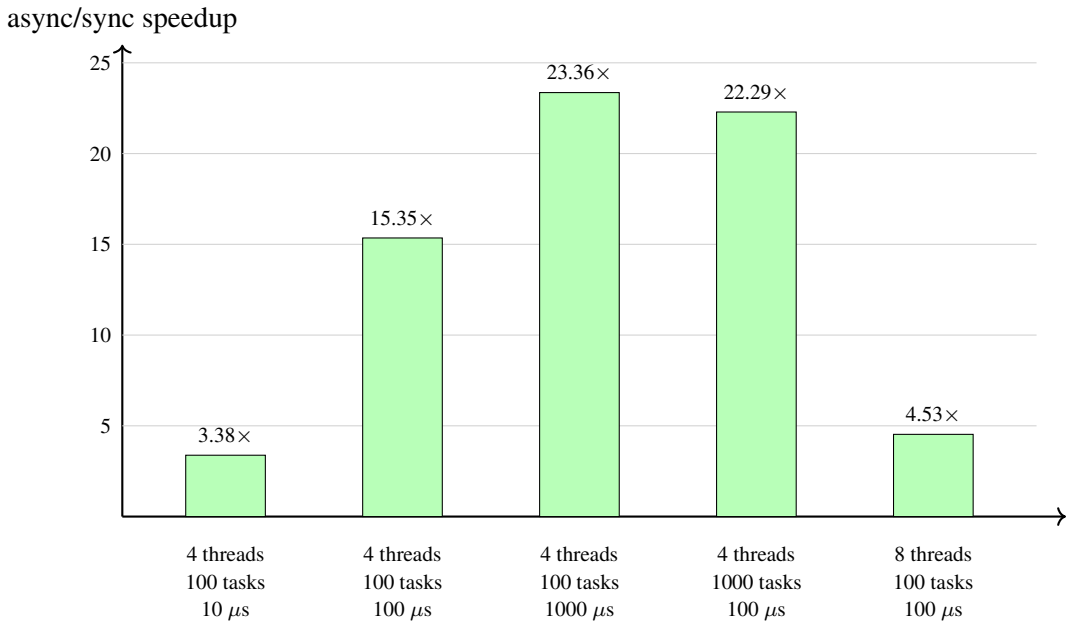

Figure 15. Relative speedup of asynchronous throughput over the synchronous baseline in representative I/O-bound configurations

Figure 15 visualises the same trend in relative form. Even when the number of worker threads is doubled to 8, the asynchronous variant remains approximately 4.5× faster at 100 μs waiting latency. This is a practically relevant result: more threads reduce part of the pressure on the synchronous baseline, but do not eliminate the structural advantage of the suspend/resume execution model.

8.6. Zero waiting latency (0ns) and coroutine scheduling overhead

With zero artificial waiting latency, the comparison no longer measures the release of worker threads from I/O waiting and instead begins to expose almost the pure cost of the suspend/resume machinery. For that reason, the most appropriate repository-backed comparison combines `BM_SyncThroughput` with a zero waiting value and `BM_AsyncYieldThroughput`, which replaces the timer-based wait with a single `PoolYield` handoff back to `ThreadPool`.

Table 22. Representative release-mode results at zero artificial waiting latency

Threads	Tasks	Synchronous throughput	Async yield throughput	Ratio async/sync
4	100	841.0k ops/s	989.4k ops/s	1.18×
4	1000	1.232M ops/s	1.184M ops/s	0.96×
8	100	810.9k ops/s	514.4k ops/s	0.63×

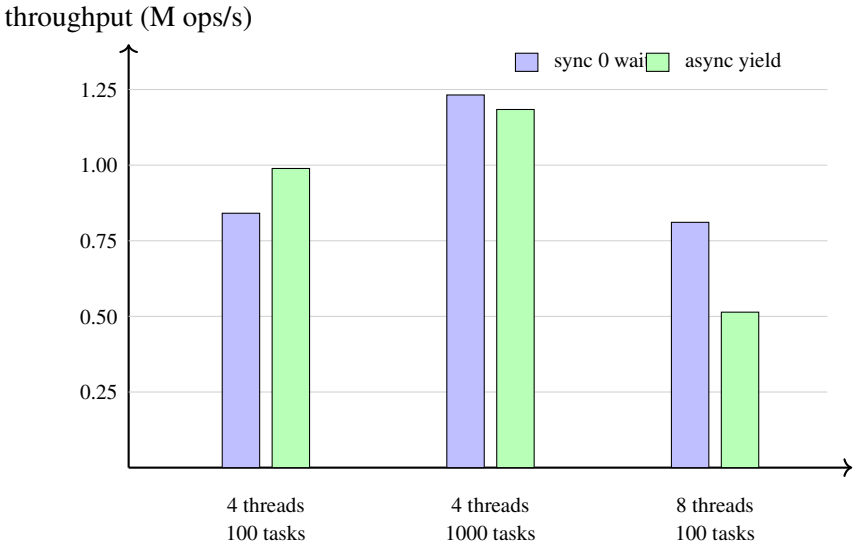

Figure 16. Throughput comparison at zero artificial waiting latency

Table 22 and Figure 16 show that, in the absence of a waiting phase, the coroutine layer no longer receives the structural bonus observed in the I/O-bound regime. With 4 threads and 100 tasks, the yield-only variant is still about 1.18× faster, but with 4 threads and 1000 tasks it is nearly at parity with the synchronous baseline (0.96×), and with 8 threads and 100 tasks it drops to 0.63×. This is a useful corrective to the strong I/O-bound results: without an external wait, the coroutine machinery does not "create" throughput, but instead adds a small yet measurable scheduling cost that can be compensated only partially.

8.7. Microbenchmark interpretation of overhead

The isolated overhead measurements show that the coroutine infrastructure is not itself the dominant cost in the I/O-bound scenarios. In the release build, `BM_TaskCreateAndWait` measures approximately 71.6 ns for creation and completion of a trivial task. `BM_ThreadPool`

`Post` remains around 5.0 μs regardless of whether the pool has 1, 2, 4, or 8 worker threads. `BM_NestedCoAwait` reaches about 324 ns at depth 10 and about 2.375 μs even at depth 100.

These numbers should not be read as a direct promised cost of every real query, but they are important for another reason. They show that the overhead of the coroutine machinery is orders of magnitude smaller than the 100–1000 μs waiting phases under which asynchronous throughput gains the most. In other words, the observed speedup is not an artefact of the measuring tool, but a consequence of worker threads no longer being locked into blocking waits.

8.8. Limitations of the empirical evaluation

The empirical evaluation deliberately uses a simulated waiting phase rather than full end-to-end execution against a concrete network database server. This is a limitation if the goal is to predict absolute throughput for a real PostgreSQL or Cassandra deployment. Yet it is also a methodological advantage if the goal is to isolate the execution model from the effects of the Structured Query Language (SQL) planner, network fluctuations, driver caching, and storage-engine specifics.

The present benchmark results therefore do not prove that “asynchronous is always faster”. They prove a narrower but more scientifically defensible claim: when the architecture operates in an I/O-bound regime and the waiting phase dominates local CPU work, a coroutine-based model allows substantially better utilisation of worker resources than the synchronous baseline. That is precisely the claim that the asynchronous layer of the library must justify.

Combined with the test-based verification, this result closes the two principal questions of the thesis. On the one hand, the library demonstrates a formal and test-supported correctness discipline. On the other hand, the asynchronous execution model demonstrates a measurable and practically relevant benefit in exactly the scenario for which it was designed. In this way the empirical evaluation does not stand apart from the architecture, but confirms its central engineering and scientific meaning.

IX. EXTENSIBILITY THROUGH NEW CONNECTORS

One of the central claims of the project is that the library is extensible without compromising its compile-time guarantees. This requires not merely the presence of tests, but a clearly defined connector contract that allows a new backend to be added without changing the query API and without breaking type discipline, capability constraints, and architectural invariants.

The previous chapters showed that the library already works across relational and non-relational backends, and that the async layer can be realised differently depending on the native capabilities of a given driver. The present chapter formalises how this is possible and how the next backend should be introduced. In other words, extensibility ceases here to be a general quality of the design and becomes a methodology with concrete requirements, signatures, capability declarations, async obligations, and completeness criteria.

9.1. Formal contract of the connector layer

A connector is described through a `DB` type and a `connector_trait<DB>` specialisation. This is a structural rather than inheritance-based contract. There is no common virtual interface such as `IConnector`. The reason is principled: different backends expose different capabilities and limitations, and those differences must remain visible at compile time. A virtual hierarchy would erase too many distinctions and would move part of validation toward runtime.

A minimally working connector must provide `wire_type`, `cursor_type`, and suitable `execute(...)` entry points. Additional abilities are denoted through capability tags. In this way the contract remains both strict and extensible. There is no “magic” registration layer in this construction: the compiler sees the specialisation directly and can verify whether it satisfies the `is_connector` concept.

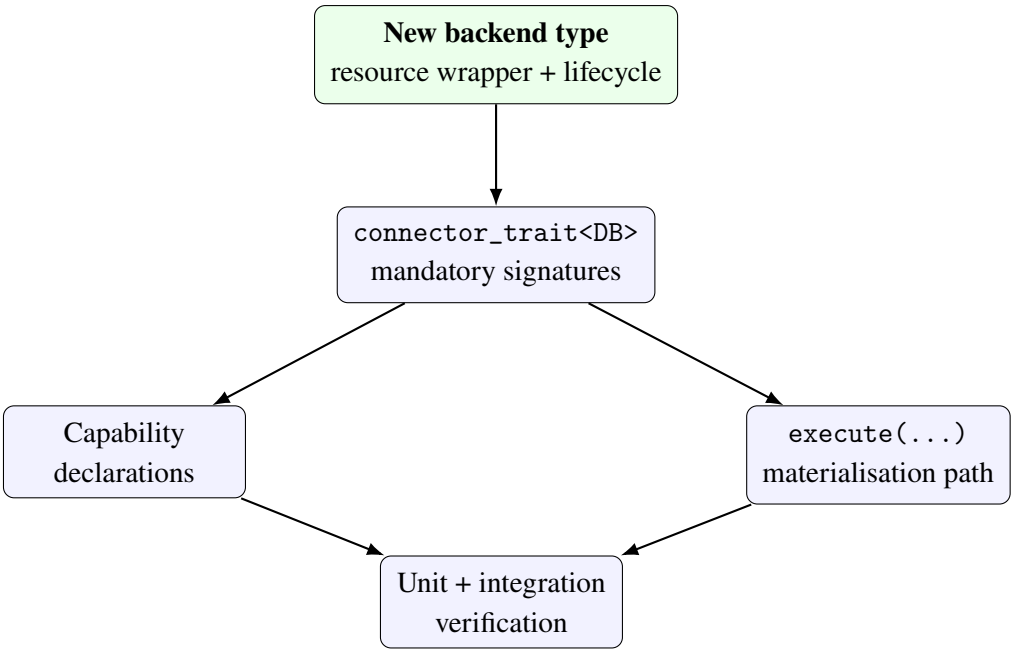

Figure 17. Architectural workflow for adding a new connector

Figure 17 shows that adding a backend is not a one-step action. It begins with a clearly defined resource wrapper, continues with the trait contract, the capability profile, and the execution materialisation, and ends only after test verification. This is essential for the thesis because it places extensibility into a disciplined engineering process.

9.2. Mandatory elements of a minimal connector

In practical terms, a backend is considered minimally integrated if:

1. a type exists that represents the backend connection wrapper;
2. a `connector_trait<DB>` specialisation exists for that type;
3. the specialisation defines `template<typenameT>structwire_type`;
4. the specialisation defines `cursor_type`;
5. the specialisation can execute at least the basic ORM query forms it claims to support.

This means that a “connector” is not merely a rendering function, but a full description of how C++ types and the query IR pass into a concrete backend. If even one of these elements is missing, the added backend is not architecturally complete, regardless of whether some subset of queries can be made to run.

Table 23. Mandatory and optional connector elements

Element	Role
`DB` type	wrapper over a backend handle, session, or mock connection
`connector_trait<DB>`	central contract specialisation
`template<typenameT>structwire_type`	maps a C++ type into its wire representation
`cursor_type`	describes result traversal or cursor lifecycle
`execute(...)` overloads	materialise the query IR into backend behaviour
Capability tags	restrict admissible operations and lifecycle policies
`begin/commit/rollback`	required when transaction support is claimed
`ddl_for(...)`	required for schema-aware integration and `migrate<DB>`
`render_constexpr(...)`	optional compile-time materialisation hook for specialised scenarios

9.3. Typing, signatures, and naming

The connector contract is strict also in terms of typing. `wire_type<T>::type` must be a stable compile-time function of `T`. `cursor_type` must model realistic result iteration or at least clearly indicate mock behaviour. `execute(...)` overloads must accept `DB&` as the first argument and return `orm::result<...>` for `SELECT` forms or `result<std::tuple<>>` for write operations.

Naming of the connectors in the repository follows a consistent convention. “Local” or unit-friendly connectors use names such as `SQLiteDB`, `PostgreSQLDB`, `MySQLDB`, `MongoDB`, `RedisDB`, `Neo4jDB`, and `CassandraDB`. Real live variants are distinguished explicitly through the suffix `LiveDB`. This is not a minor stylistic choice, but part of contract clarity: from the type name alone it is clear whether a given connector is a mockable adapter, an in-memory/local wrapper, or a real driver-backed backend.

Absence of inheritance must also be regarded as a formal rule rather than an implementation detail. A new connector should not derive from a common “ORM base class”, because that would mix backend-specific logic with runtime polymorphism. The correct extension mechanism is specialisation of `connector_trait<DB>`.

9.4. Capability mechanism and compile-time restrictions

Capability tags are the primary way in which the library distinguishes backends by admissible operations. If `connector_trait<DB>` does not declare `supports_joins`, then an attempt to execute a join-based query is architecturally invalid and should be rejected already at compilation. The same reasoning applies to transactions, aggregation, and concurrent execution.

From the standpoint of verification, this is important because correctness no longer depends solely on tests. Part of the contract is directly checkable by the compiler. The tests then no longer prove that the contract exists, but that concrete specialisations apply it consistently. This is exactly why `tests/unit/test_mongodb_connector.cpp` and `tests/unit/test_redis_connector.cpp` validate the absence of `supports_joins`, `supports_transactions`, and `supports_aggregation` for those connectors.

This negative verification is just as important as positive verification. Proving that a backend *does not* support a given operation is as important as proving that it does support another.

9.5. Minimal connector skeleton and operational connector skeleton

In the repository, `MockDB` and `MockMigrateDB` are especially valuable because they act as connector archetypes. The former shows the operational contract for query execution, while the latter shows how the same contract can be extended toward schema-aware migration scenarios.

```
template <>
struct connector_trait<MockDB>
{
    using supports_joins              = void;
    using supports_transactions       = void;
    using supports_aggregation        = void;
    using supports_upsert             = void;
    using supports_bulk_insert        = void;
    using supports_concurrent_execute = void;

```

```
    template <typename T>
    struct wire_type { using type = T; };

    struct cursor_type
    {
        [[nodiscard]] bool has_next() const noexcept { return false; }
    };

    template <typename Response, typename Joins, typename Wheres,
              typename Limits, typename Groups, typename Orders>
    static auto execute(MockDB& db,
        select_query<Response, Joins, Wheres, Limits, Groups, Orders> q)
        -> result<projected_type<Response>, Response>;
};
```

Listing 11: Excerpt from `connector_trait<MockDB>`

Listing 11 is revealing because it gathers in compact form almost all mandatory elements: capability profile, `wire_type`, `cursor_type`, and query-specific `execute(...)` signatures. Moreover, the connector uses separate overloads for `select_query`, `insert_query`, `update_query`, and `delete_query`, making the mapping toward backend behaviour explicit.

```
template <>
struct connector_trait<MockMigrateDB>
{
    using supports_joins              = void;
    using supports_transactions       = void;
    using supports_aggregation        = void;
    using supports_concurrent_execute = void;

    template <typename T>
    struct wire_type { using type = T; };

    template <typename... Args>
    static auto execute(MockMigrateDB& db, Args&&... args)
    {
        return connector_trait<MockDB>::execute(
            static_cast<MockDB&>(db), std::forward<Args>(args)...);
    }

    [[nodiscard]] static std::string ddl_for(const create_table_op& op);
    [[nodiscard]] static std::string ddl_for(const add_column_op& op);
};
```

Listing 12: Excerpt from the schema-aware extension `MockMigrateDB`

Listing 12 shows how the operational connector skeleton can be extended. Query-execution logic is delegated to `MockDB`, but DDL hooks and transaction hooks are

added. This is a good example of a progressive connector-maturity model.

9.6. Transactions, thread safety, and prepared queries

A mature backend contract is not exhausted by a single `SELECT`. If the connector declares `supports_transactions`, it is expected to support `begin`, `commit`, and `rollback` so that `transaction_guard<DB>` remains valid. If it declares `supports_concurrent_execute`, it must participate safely in `connection_pool<DB,N>`.

The prepared-query layer also imposes requirements regarding the stability of `DB&` lifetime and the consistency of binding behaviour under repeated execution. Consequently, a mature connector should be evaluated not only by whether it renders correct syntax, but also by the operational contract around resources, transactions, and reuse. This is particularly important for backends whose driver lifecycle is more complex than that of `MockDB`.

9.7. Migrations and schema-aware integration

A connector that participates in `migrate<DB>` must also offer DDL extension points. In the repository this is demonstrated through `MockMigrateDB`. The example is especially useful because it shows how schema-aware behaviour can be layered on top of an already existing execution backend. In this way, the library formalises different levels of connector completeness.

It is important to note that schema-aware support is not mandatory for every backend. Some connectors may be entirely adequate for query-execution analysis without supporting automated migrations. But if a connector claims a fuller ORM integration, the presence of `ddl_for(...)` hooks and migration tests becomes mandatory.

Figure 18. Maturity ladder for a new connector

Figure 18 proposes a practical evaluation model. It is useful also for future development of the library because it allows new backends to be positioned not simply as “connector present/absent”, but according to the level of their integration.

9.8. Methodology for adding a new backend

Adding a new backend follows a clear sequence. First, the backend type and its resource wrapper are defined. Next, `connector_trait<DB>` is implemented with the mandatory elements. Then capability declarations and the rules for rendering or executing the query IR are added. Finally, unit tests for the contract are written and, when possible, live integration tests are added for the real driver.

The architecturally correct order is important: one should not begin from runtime workarounds, but from formalising the compile-time boundaries and the mapping logic. If this order is violated, the result is usually a connector that “works” for one happy path but is not actually compatible with the general architecture.

Table 24. Recommended verification matrix for a new connector

Verification layer	What must be proven	Representative repository correlate
Concept compliance	`is_connector<DB>` is satisfied	`tests/unit/test_mongodb_connector.cpp`
Capability honesty	declared and undeclared capability tags are correct	`tests/unit/test_redis_connector.cpp`, `tests/integration/test_sqlite.cpp`
Execution correctness	the query IR is materialised correctly	`tests/integration/test_mockdb.cpp` and the live backend tests
Resource/lifecycle correctness	transaction and concurrency hooks behave correctly	`tests/unit/test_thread_safety.cpp`
Schema-aware correctness	DDL hooks and migration diff behave correctly	`tests/unit/test_schema_migration.cpp`

9.9. Criteria for a “fully implemented and functioning” connector

On the basis of the code and tests in the repository, the following criteria can be formulated:

1. the connector satisfies the `is_connector` contract;
2. capability declarations correspond to real behaviour;
3. naming and typing conventions remain compatible with the other connectors;
4. resource lifecycle is correct and testable;
5. supported ORM operations are covered by unit tests;
6. when a live driver exists, integration tests run against a real database;

7. when schema-aware support is claimed, DDL hooks and migration tests are present.

These criteria are stricter than merely saying “there is a working adapter”, because they require consistency between architectural intent, API contract, and verified behaviour. That is precisely what distinguishes an ad hoc integration from a connector that truly belongs to the library.

9.10. Verification through tests

The test layer in the repository follows the structure of the architecture. Unit tests for the individual connectors validate `is_connector` compliance, capability declarations, and basic rendering. `tests/unit/test_thread_safety.cpp` checks `connection_pool`, `thread_local_db`, and `transaction_guard`. `tests/unit/test_schema_migration.cpp` validates the migration diff logic. `tests/integration/test_mockdb.cpp` and the live integration tests for individual backends prove that the abstraction also works during real execution.

Verification is therefore not concentrated in one place. It is distributed so that every architectural claim has at least one direct testing correlate. This turns connector extensibility from a declarative objective into a verifiable engineering practice.

9.11. Assessment of the existing connectors

`SQLiteDB` may be regarded as a reference mature implementation for a relational backend. `PostgreSQLDB`, `MySQLDB`, `MongoDB`, `RedisDB`, `Neo4jDB`, and `CassandraDB` already demonstrate important parts of the contract and have varying degrees of live coverage. `MockDB` and `MockMigrateDB` play a special role as verification connectors: they are not merely test doubles, but instruments for formalising the rules of the architecture.

A broader conclusion follows from this: the extensibility of the library was not added afterwards, but is embedded in the very way the connector layer is defined. That is exactly what allows the project to continue toward new backends without breaking the formal character of the compile-time ORM architecture.

X. LIMITATIONS AND FUTURE WORK

The current version of the library already demonstrates a working architecture for Compile-time Object-Relational Mapping across relational and non-relational backends, but scientific accuracy requires a clear distinction between what has already been achieved and what remains for future work. The repository contains a real query-IR layer, a capability-based connector contract, a migration prototype, thread-safety helpers, and a substantial set of unit and integration tests. At the same time, some subsystems still need to be extended or validated more broadly under production-like conditions.

Future development is not an arbitrary list of ideas. It follows directly from the architectural decisions analysed in the previous chapters. If the C++ model is the primary carrier of the logical schema, then fuller schema-aware tooling is a natural next step. If the connector layer is the central point of extensibility, then future work must refine the taxonomy of capabilities. If the query IR is truly the shared layer across distinct backends, then it must be exercised under richer operational scenarios and stricter performance constraints.

10.1. Strategic directions of development

From the perspective of the present thesis, future work can be grouped into four strategic directions. The first is *greater operational maturity* of the already existing backends. The second is *more complete automation of schema and metadata handling*. The third is *reduction of runtime overhead* through lower-level execution and tighter control of the wire-protocol layer. The fourth is *reduction of boilerplate* through deeper integration with newer C++ language facilities.

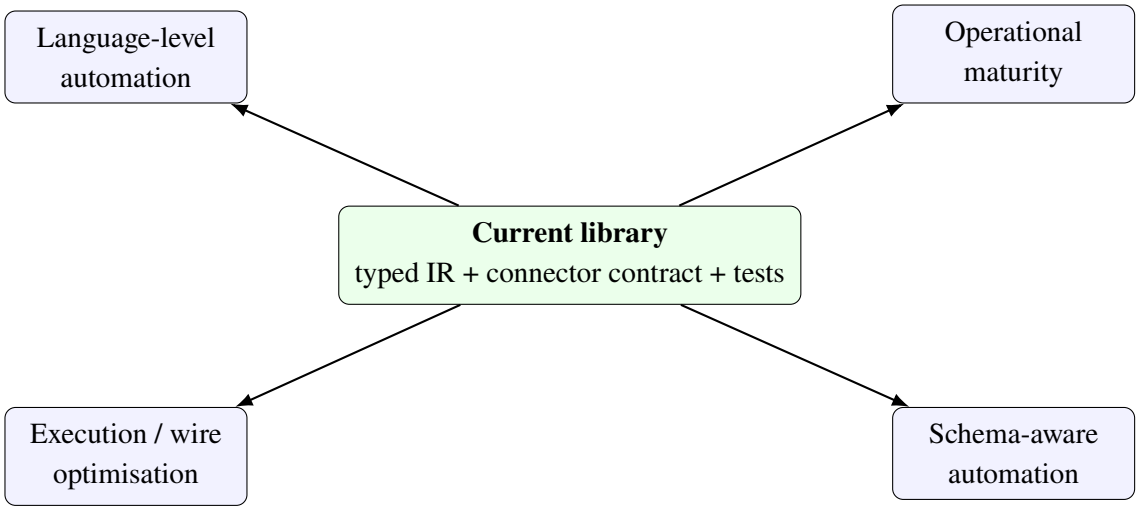

Figure 19. Four strategic directions for future development

Figure 19 is useful because it shows that future work is not one-dimensional. Some tasks aim at operational maturity, others at stronger compile-time automation, and still others at performance and runtime engineering. This distinction matters because the different directions require different success criteria.

10.2. Broader live backend coverage

Although the repository already contains live integration tests for several backends, the maturity of the individual implementations is not identical. SQLite remains the most natural reference relational path because it combines query rendering, binding logic, and result hydration most tightly. For PostgreSQL, MySQL, MongoDB, Redis, Neo4j, and Cassandra, the next step is broader coverage of operational scenarios: richer type families, diagnostic paths under failure, connection lifecycle policies, and behaviour under more demanding query patterns.

This direction matters not only as an engineering improvement, but also as a scientific validation of the limits of the shared query IR. Only broader empirical coverage can show how far one and the same logical layer remains adequate across distinct storage systems. A join-oriented logic that is natural for PostgreSQL, for instance, carries a very different operational meaning in Neo4j or Cassandra. Live coverage is therefore not merely a testing discipline, but an instrument for studying architectural applicability.

In addition, live backend coverage should include negative paths: connection-establishment failures, partially invalid parameter sets, driver-specific timeout scenarios, and behaviour under interrupted connections. These questions are often underestimated in early ORM systems, but in a multi-backend context they are essential for real adoption.

10.3. Full schema-aware integration and runtime introspection

The prototype migration layer is sufficient to show how the C++ entity model can participate in schema evolution. The next natural direction is live schema introspection, metadata extraction from drivers, and safe execution of DDL operations. This is especially important for relational backends, but it also matters for document and wide-column systems, where the notion of schema may be partial or more flexible.

Further work in this direction must clarify the boundary between compile-time

guarantees and runtime facts about a deployed system. The compiler can validate query structure, but it cannot validate the real state of a production schema without supplementary runtime information. A future migration layer should therefore combine two sources of truth: the entity description in C++ and the metadata picture extracted from the live database.

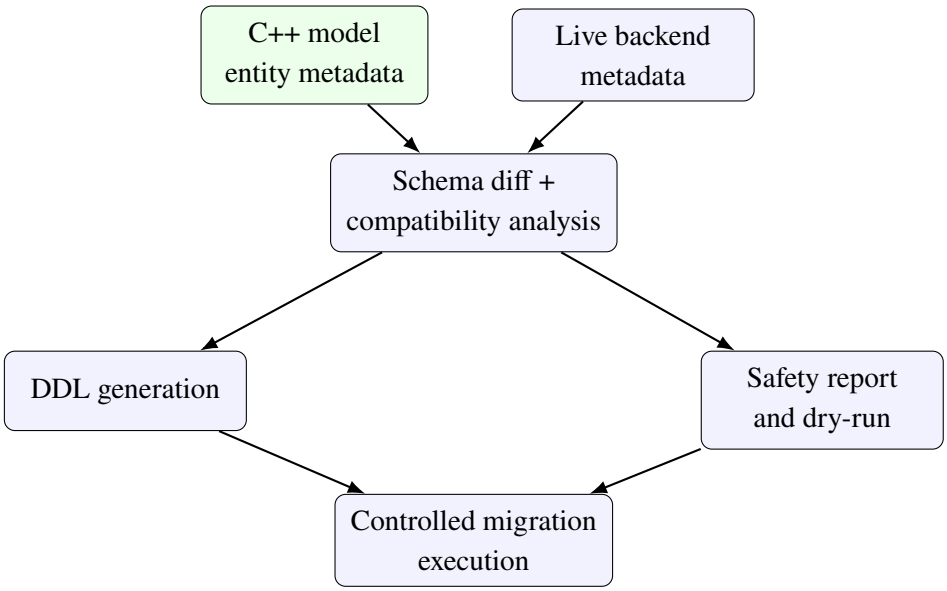

Figure 20. Target direction for schema-aware tooling beyond the current migration prototype

Figure 20 shows that the future migration pipeline must be bidirectional: it must read both from the compile-time description and from the live system. This is the natural next step if the library is to be used not only experimentally, but also as a practical development tool.

10.4. Thread safety and connection lifecycle

The existing helper layer for connection pooling, thread-local access, and transaction guards already provides an architectural basis for multi-threaded use. Future work must answer more precise questions: how exactly `supports_concurrent_execute` should be defined for different drivers; when a connection is safe to share; and which policies should govern reconnection, shutdown, and deferred release of resources.

There is also an important research dimension here: different backends have different concurrency semantics, and therefore compile-time capability annotation must be precise enough to avoid misleading promises. It is entirely possible that future versions of the library will split the current `supports_concurrent_execute` into finer-grained capabilities such as safe parallel reads, parallel execution with isolated connection handles, or strict serialisation policies.

In practical terms, this means that the thread-safety helper layer should not remain an isolated convenience module. On the contrary, it should become an architectural continuation of the connector contract and participate in the full assessment of backend maturity.

10.5. Low-level execution and wire-protocol optimisation

The experimental layer in `WireProtocol/wire_protocol.hpp` points toward future directions such as `io_uring` / IOCP awaitables, `zero_copy_result`, `batch_insert`, and `make_constexpr_sql`. These components are not yet a production-ready communication stack, but they outline a possible path by which the library may move beyond the classical synchronous driver model.

Especially promising are zero-copy result handling and compile-time SQL generation for connectors that can support it. Both would reduce runtime overhead further without sacrificing the typed architecture. This matters because a common criticism of heavily typed abstraction layers is that they hide excessive runtime cost. Future work should demonstrate that compile-time expressiveness and low overhead are not necessarily opposing goals.

Moreover, performance should not be viewed only through latency or throughput. It also includes memory layout of results, the number of temporary allocations, repeated reuse of prepared-query artefacts, and the quality of driver-level batching strategies.

10.6. Integration with C++26 reflection and further automation

Support for C++26 reflection has already been conceptually prepared through build-time detection and helper paths, and `tests/unit/test_cpp26_reflection.cpp` shows the presence of an early verification route. The public API, however, still relies on explicit string literal names in `property<T,Name>`. Future work should turn the reflection path into a stable public mechanism that can infer field names automatically while preserving backward compatibility.

Such a development would reduce boilerplate and make even more explicit the idea that the C++ model is the primary source of the logical schema. From a research perspective, it would also move the library closer to a model-first style in which much of the metadata is derived automatically rather than duplicated manually.

There is, however, an important boundary here. Reflection should not weaken the clarity of the compile-time contract. If automation reduces type transparency or makes compiler

diagnostics harder to understand, it may undermine one of the principal values of the library. This direction must therefore be pursued carefully.

10.7. Extension of the connector contract and capability taxonomy

As new backends are added, it may become necessary to distinguish more finely between classes of capability. Future versions of the library may need to separate graph traversal support, partial document embedding support, schema introspection support, streaming result support, or explicit batching support. This direction follows naturally from the current trait-based architecture.

An important rule must be preserved, however: new capability tags should never be decorative. They must carry real compile-time meaning and constrain inadmissible operations. If a capability tag has no architectural consequence, it adds noise instead of value and makes the contract less precise.

Table 25. Priority directions for future development

Direction	Affected subsystems	Expected effect	Principal challenge
Live backend maturity	connectors, integration tests, error paths	higher practical reliability	different operational semantics across backends
Schema-aware automation	`migrate<DB>`, DDL hooks, metadata introspection	a more production-oriented ORM integration	combining compile-time and runtime truth
Execution optimisation	wire-protocol layer, prepared queries, batching	lower runtime overhead	preserving the typed abstraction
Reflection-driven modelling	entity layer, metadata extraction, API ergonomics	less boilerplate and a more automatic model-first workflow	balancing automation against transparency
Capability refinement	connector contract and static checks	a more precise contract for future backends	avoiding decorative capability tags

10.8. Need for stricter empirical evaluation

A future version of the work should also include a more systematic benchmarking layer. It should compare not only separate backends, but also different execution modes inside the library itself — direct execution, `prepare()` + reuse, batch-insert scenarios, and distinct forms of result materialisation. Such benchmarking would be valuable both engineering-wise and scientifically because it would reveal the concrete cost of the different architectural choices.

Alongside this, it would be useful to add a formalised evaluation of error diagnostics. One of the strengths of a compile-time ORM design lies precisely in early fault localisation. The quality of error messages and the clarity of compile-time failure paths are therefore themselves valid objects of investigation.

10.9. Summary

The future of the library does not begin from an empty slate. On the contrary, it builds on an architecture that is already general enough to absorb new backends and strict enough to enforce formal limits. This is what makes the work suitable both for continued engineering development and for further academic investigation into compile-time modelling of data access.

The most important conclusion of the present chapter is that future work should not be understood as a repair of an incoherent prototype. It is the natural unfolding of an already defensible architectural foundation. That difference matters: the library has reached a stage at which extension is a matter of disciplined development rather than conceptual reinvention.

XI. CONCLUSION

This thesis presented the design, implementation, and analysis of a Compile-time Object-Relational Mapping library for C++, intended for both relational and non-relational databases. At the centre of the work stands the idea that query structure, the logical data model, and a significant portion of backend admissibility constraints can be described and validated through the type system of the language.

In doing so, the thesis addresses a concrete problem of contemporary practice: the mismatch between strongly typed application code and the heterogeneous storage models that such code must use. Classical ORM solutions are strongest within the relational world, while purely driver-oriented approaches often move too much correctness toward runtime. The present work showed that between these two poles there exists a viable architectural position — a compile-time ORM layer that is practical, formally organised, and sufficiently honest about backend differences.

11.1. Summary of the solved problem

The task was not merely to build another convenient query builder. The goal was to design a library in which:

1. the logical data model is described through C++ types;
2. queries are represented as a typed IR rather than as strings;
3. backends are integrated through a formal connector contract;
4. differences between storage models are managed through capability and constraint mechanisms;
5. correctness is supported simultaneously by compile-time checks and by a testing infrastructure.

Within the thesis, each of these sub-goals was addressed through concrete architectural and implementation decisions. The entity layer introduced typed properties and relationships, the `store_as` policy, and traits for logical container naming. The query builder was organised around a typed intermediate representation. The connector layer was formalised through `con`

nector_trait<DB>, and the verification strategy was supported by unit and integration tests across multiple backends.

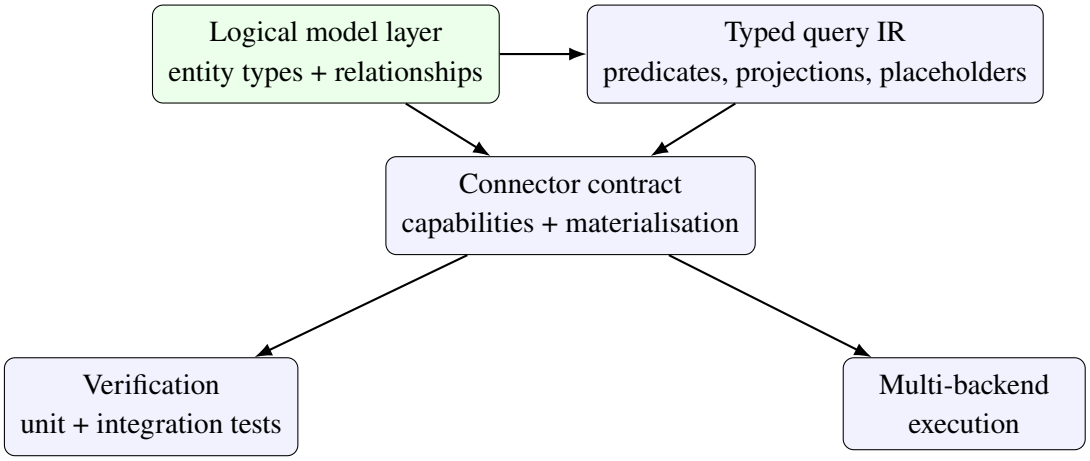

Figure 21. Synthesis of the principal construction of the developed library

Figure 21 summarises the overall logic of the thesis. It shows that the library is not a collection of isolated techniques, but a coherent system in which the model layer, query IR, connector contract, verification, and multi-backend execution support one another.

11.2. Summary of achieved results

The work showed that C++ provides sufficiently strong mechanisms — templates, `const expr`, `concepts`, and trait specialisations — for building an ORM layer in which queries are not described as strings. Instead, they are represented through a typed query IR that can be analysed at compile time and materialised differently depending on the chosen backend.

At the level of data modelling, the library demonstrates how `property`, `relationship`, `store_as`, and `table_name_trait` can serve as a unified logical layer. That layer is not restricted to relational interpretation, but acts as a shared description of data and relationships. It is precisely this that allows one and the same application-level structure to be carried toward tables, documents, key-value structures, graph nodes, or wide-column rows.

At the level of architecture, the connector trait is formalised as the central extensibility mechanism. Through it, the library adapts one and the same query IR to SQL, BSON-like filters, Redis commands, Cypher, and CQL. The capability mechanism complements that architecture by making inadmissible operations visible already at compilation.

At the level of verification, the work relies on a systematic set of unit and integration tests covering the entity layer, the query builder, prepared queries, migrations, thread safety, and

concrete connectors. Architectural claims are therefore supported by real evidence from code and testing infrastructure.

11.3. Scientific and practical contributions

The first contribution of the work is the demonstration that a practically useful Compile-time Object-Relational Mapping library can be realised in standard C++. This is achieved without external code generation, without a virtual connector interface, and without forcing the user outside idiomatic C++.

The second contribution lies in clarifying the relationship between the C++ entity description and the physical organisation of stored data. The thesis shows that the C++ model can be treated as the primary carrier of the logical schema, while the concrete backend determines the mode of physical materialisation. This claim has value not only for ORM libraries, but more broadly for systems that try to combine strong static typing with heterogeneous data sources.

The third contribution is the formalisation of a connector contract that combines extensibility with strict compile-time discipline. This contract makes it possible to extend the library with new backends without breaking the guarantees already embedded in the query API. It is exactly here that the work differs from approaches that provide a shared interface but conceal critical backend differences.

The fourth contribution is the demonstration that verification of such a library must be multi-layered. It is not sufficient merely to “have tests” or merely to “have compile-time checks”. What is needed is a combination of the two: concept-level constraints, capability assertions, query-rendering tests, live integration scenarios, and migration/thread-safety validation.

Table 26. Relationship between the thesis goals and the realised results

Goal	Realised mechanism	Evidence basis
Typed data model	`property`, `relationship`, `table_name_trait`	entity headers and the property/relationship unit tests
Typed query layer	query builders and typed IR	query headers, `tests/unit/test_query.cpp`, `tests/integration/test_mockdb.cpp`
Extensible multi-backend layer	connector contract and capability tags	connector headers and backend-specific unit/live tests
Schema-aware direction	`migrate<DB>` and DDL hooks	`tests/unit/test_schema_migration.cpp`
Safe operational use	thread-safety and transaction helpers	`tests/unit/test_thread_safety.cpp`

Table 26 shows that the thesis does not remain at the level of a declarative design document. For each principal goal there is a concrete realised mechanism and at least one direct technical correlate in the repository.

11.4. Principal limitations and the boundaries of generalisation

The work also shows clearly that a multi-backend ORM abstraction has limits. Not every storage model offers the same operations, and therefore the shared query IR cannot be interpreted identically everywhere. That is precisely why capability and constraint mechanisms are so important. They do not weaken the library; they make it correct with respect to different backend families.

In addition, although the architectural foundation for migrations, thread safety, and lower-level execution support is already present, these subsystems remain natural areas for future development and broader empirical expansion. This means that the work should be evaluated as a completed architectural and implementation demonstration, but not as the last word on compile-time data access.

From an academic point of view, it is also important to stress another boundary. The

present work does not prove that compile-time ORM is universally the best approach for all applications. It proves something more precise: that for a class of systems requiring clear static structure, high type safety, and controlled multi-backend extensibility, this approach is viable and engineering-sound.

11.5. Significance for practice and for future research

The practical value of the work lies in the fact that the library already provides a stable foundation for further engineering development. The entity description, query IR, connector contract, and verification strategy are sufficiently clear to allow disciplined addition of new backends and operational capabilities.

Its research value is related to the broader question of the role of the type system in modelling data access. The thesis shows that the C++ type system can serve not only for describing data and algorithms, but also for formalising architectural boundaries, admissibility of operations, and the transition between logical and physical layers. This places the library in a wider context that extends beyond ORM in the narrow sense.

11.6. Closing remarks

The analysis confirms that the C++ type system is sufficiently powerful to serve not only for modelling data and algorithms, but also for formalising access to persistent systems. The combination of a static entity model, query IR, a trait-based connector layer, and systematic verification demonstrates a viable path toward libraries that are expressive, safe, and extensible at the same time.

In this way, the thesis achieves both an engineering and a research goal: it delivers a working library while also formulating principles that can guide future development of compile-time data-access layers. The most important conclusion is that the compile-time ORM approach is not merely a theoretical exercise, but a real engineering alternative for systems that require rigour, clarity, and extensibility across multiple storage models.

REFERENCES

[1] Apple Inc. kqueue(2) — kernel event notification mechanism. https://developer.apple.com/library/archive/documentation/System/Conceptual/ManPages_iPhoneOS/man2/kqueue.2.html, 2009. Manual page.

[2] Code Synthesis Tools CC. ODB: C++ object persistence framework. https://www.codesynthesis.com/products/odb/, 2010. Online documentation.

[3] DataStax. DataStax C/C++ driver: Futures. https://datastax.github.io/cpp-driver/2.6/topics/basics/futures/, 2026. Online documentation, accessed 2026-04-06.

[4] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[5] Google. Google Benchmark user guide. https://google.github.io/benchmark/user_guide.html, 2026. Online documentation, accessed 2026-04-06.

[6] D. Richard Hipp. SQLite: A self-contained, serverless, zero-configuration, transactional SQL database engine. https://www.sqlite.org/, 2000. Online documentation.

[7] D. Richard Hipp. SQLite C/C++ interface – an introduction. https://www.sqlite.org/cintro.html, 2000. Online documentation.

[8] Maciej Hryniuk and Mateusz Bao. SOCI: The c++ database access library. https://soci.sourceforge.net/, 2004. Online documentation.

[9] ISO/IEC JTC1/SC22/WG21. P1063R0: Core coroutines. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1063r0.pdf, 2018. ISO/IEC JTC1/SC22/WG21 proposal.

[10] ISO/IEC JTC1/SC22/WG21. P0912R5: Merge coroutines TS into C++20 working draft. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0912r5.html, 2019. ISO/IEC JTC1/SC22/WG21 proposal.

[11] ISO/IEC JTC1/SC22/WG21. ISO/IEC 14882:2020 – programming language C++. International standard, International Organization for Standardization, 2020.

[12] ISO/IEC JTC1/SC22/WG21. ISO/IEC 14882:2024 – programming language C++. International standard, International Organization for Standardization, 2024.

[13] Chris Leishman. libneo4j-client — C client library for Neo4j. https://neo4j-client.net/, 2026. Online documentation, accessed 2026-04-06.

[14] MongoDB, Inc. libmongoc: mongoc_client_pool_t. https://mongoc.org/libmongoc/current/mongoc_client_pool_t.html, 2026. Online documentation, accessed 2026-04-06.

[15] Oracle Corporation. MySQL connector/C – C client library for MySQL. https://dev.mysql.com/doc/c-api/en/, 2000. Online documentation.

[16] Oracle Corporation. MySQL C api developer guide: C api asynchronous interface. https://dev.mysql.com/doc/c-api/8.4/en/c-api-asynchronous-interface.html, 2026. Online documentation, accessed 2026-04-06.

[17] Redis Ltd. and hiredis contributors. hiredis — minimalistic C client library for Redis. https://github.com/redis/hiredis, 2026. GitHub repository and documentation, accessed 2026-04-06.

[18] The PostgreSQL Global Development Group. libpq — C library for PostgreSQL. https://www.postgresql.org/docs/current/libpq.html, 1996. Online documentation.

[19] The PostgreSQL Global Development Group. PostgreSQL documentation: Asynchronous command processing in libpq. https://www.postgresql.org/docs/current/libpq-async.html, 2026. Online documentation, accessed 2026-04-06.

[20] Roland Witt. sqlpp11: A type safe embedded domain specific language for SQL queries and results in C++. https://github.com/rbock/sqlpp11, 2014. GitHub repository.